

数据结构与算法

查找

查找

8.1 查找的基本概念

8.2 静态表的查找

8.2.1 顺序查找

8.2.2 有序表的查找

8.3 动态查找表

8.3.1 二叉排序树

8.3.2 二叉平衡树

8.3.3 B树和B+树

8.4 散列表

8.4.1 散列表的概念

8.4.2 构造散列函数的方法

8.4.3 处理冲突的方法

8.1 查找的基本概念

- 查找表是由同一类型的数据元素(也可称为记录)构成的集合。
- 由于“集合”中的数据元素之间存在着松散的关系, 因此查找表是一种应用灵便的结构。
- 在每个数据元素中可能存在有若干数据项, 可以标识数据元素的数据项称为关键字。

8.1 查找的基本概念

- 关键字是数据元素中的某个数据项。
- 唯一能标识数据元素（或记录）的关键字，值互不相同，我们称这种关键字为主关键字。
- 若查找表中某些元素的关键字值相同，称这种关键字为次关键字。例如，银行帐户中的帐号是主关键字，而姓名是次关键字。

8.1 查找的基本概念

基本操作：

- (1) 在查找表中查看某个特定的数据元素是否在查找表中。
- (2) 检索某个特定元素的各种属性。
- (3) 将查找表中不存在的数据元素插入。
- (4) 从查找表中删除某个数据元素。

8.1 查找的基本概念

- **静态查找表**：在查找过程中查找表本身不发生变化。对静态查找表进行的查找操作称为静态查找。
- **动态查找表**：在查找过程中查找表可能会发生变化。对动态查找表进行的查找操作称为动态查找。

8.1 查找的基本概念

- 查找：在数据元素集合中查找满足某种条件的数据元素的过程称为查找。
- 最简单且最常用的查找条件是“关键字值等于某个给定值”，在查找表搜索关键字等于给定值的数据元素（或记录）。若表中存在这样的记录，则称查找成功，此时的查找结果应给出找到记录的全部信息或指示找到记录的存储位置。

8.1 查找的基本概念

- 若表中不存在关键字等于给定值的记录，则称查找不成功，此时查找的结果可以给出一个空记录或空指针。
- 若按主关键字查找，查找结果是唯一的；若按次关键字查找，结果可能是多个记录，即结果可能不唯一。

8.1 查找的基本概念

- 查找表的存储结构：查找表是一种非常灵活的数据结构，对于不同的存储结构，其查找方法不同。为了提高查找速度，有时会采用一些特殊的存储结构。
- 本章将介绍以线性结构、树型结构及哈希表结构为存储结构的各种查找算法。

8.1 查找的基本概念

- 查找的基本方法可以分为两大类，即比较式查找法和计算式查找法。
- 比较式查找法又可以分为基于线性表的查找法和基于树的查找法。
- 而计算式查找法也称为HASH（哈希）查找法。

8.1 查找的基本概念

- 查找算法的时间效率：查找过程的主要操作是关键字的比较，所以通常以“平均查找长度”来衡量查找算法的时间效率。
- 平均查找长度：为确定数据元素在列表中的位置，需和给定值进行比较的关键字个数的期望值，称为查找算法在查找成功时的平均查找长度。

8.1 查找的基本概念

对于长度为n的列表，查找成功时的平均查找长度为

$$ASL = P_1C_1 + P_2C_2 + \cdots + P_nC_n = \sum_{i=1}^n P_iC_i$$

其中 P_i 为查找列表中第 i 个数据元素的概率， C_i 为找到列表中第 i 个数据元素时，已经进行过的关键字比较次数。由于查找算法的基本运算是关键字之间的比较操作，所以可用平均查找长度来衡量查找算法的性能。

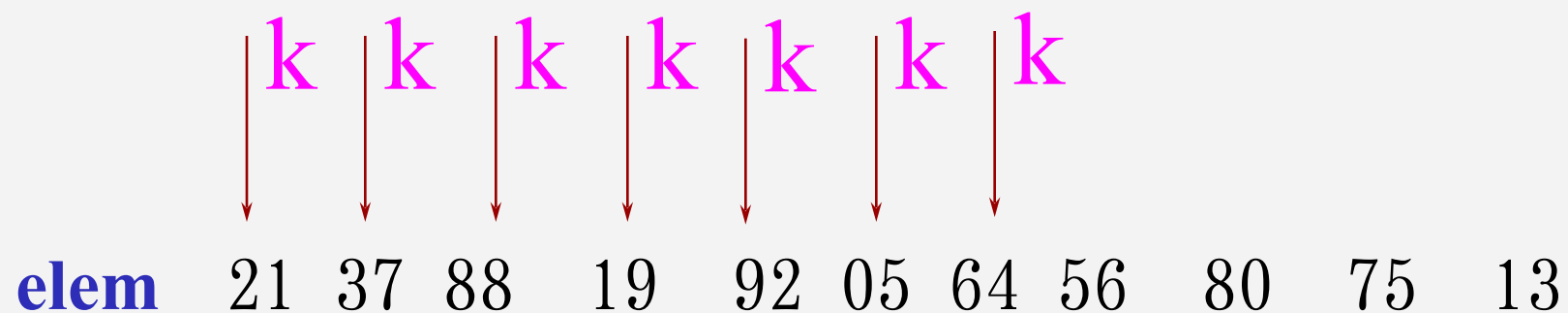
8.2 静态表的查找

- 静态查找是指在静态查找表上进行的查找操作，在查找表中查找满足条件的数据元素的存储位置或各种属性。
- 静态查找表可以有不同的表示方法，在不同的表示方法中，实现查找操作的方法也不同。
- 本节将讨论以线性结构表示的静态查找表及相应的查找算法。

8.2.1 顺序查找

- 顺序查找是一种最简单的查找方法。其基本思想是将查找表作为一个线性表，可以顺序查找，也可按关键字值进行查找。在查找过程中，给定值与表中每个元素的关键字值进行比较，若某个记录的关键字值与给定值相等，则查找成功，返回该记录的存储地址；反之，若直到最后一个记录，其关键字值均不等于给定值，则查找失败，返回查找失败标志。

8.2.1 顺序查找



假设给定值 $key=64$,
要求 $elem[k] == key$, 问: $k = ?$

8.2.1 顺序查找

- 用平均查找长度来分析一下顺序查找算法的性能。假设列表长度为 n ，那么查找第 i 个数据元素时需进行 $i+1$ 次比较，即 $C_i=i+1$ 。又假设查找每个数据元素的概率相等，即 $P_i=1/n$ ，则顺序查找算法的平均查找长度为

$$ASL = \sum_{i=0}^{n-1} P_i C_i = \frac{1}{n} \sum_{i=0}^{n-1} C_i = \frac{1}{n} \sum_{i=0}^{n-1} (i+1) = \frac{1}{2} (n+1)$$

8.2.2 有序表的查找

- 前面顺序查找表的查找算法简单，但平均查找长度较大，特别不适用于表长较大的查找表。为了改进不足，从数据排列结构进行改善。
- 以有序表表示静态查找表时，查找函数可用折半查找来实现。

8.2.2 有序表的查找

- 折半查找法又称为二分法查找法，这种方法要求待查找的列表必须是按关键字大小有序排列的顺序表。
- 折半查找要求查找表用顺序存储结构存放且各数据元素按关键字有序（升序或降序）排列，也就是说折半查找只适用于对有序顺序表进行查找。

8.2.2 有序表的查找

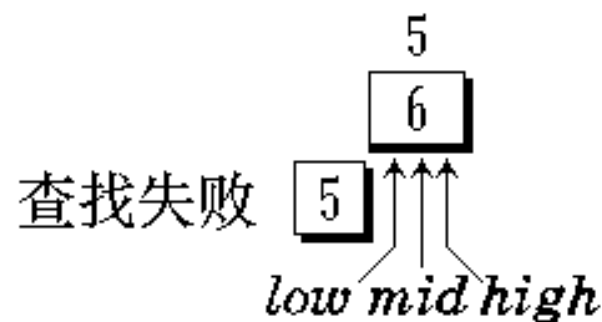
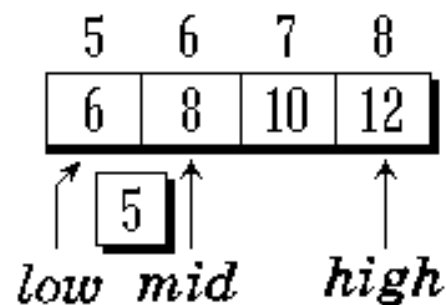
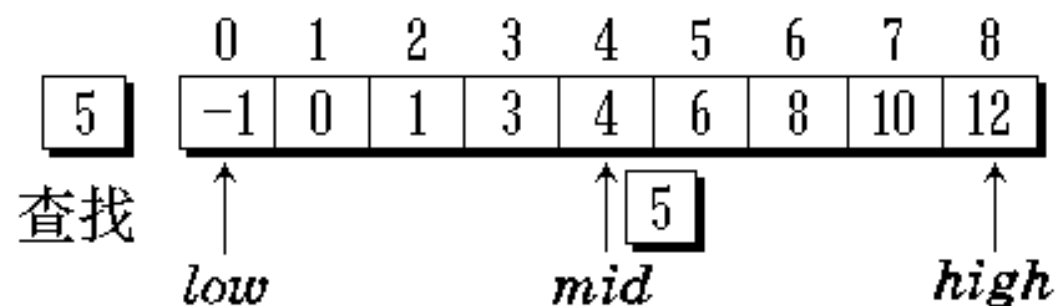
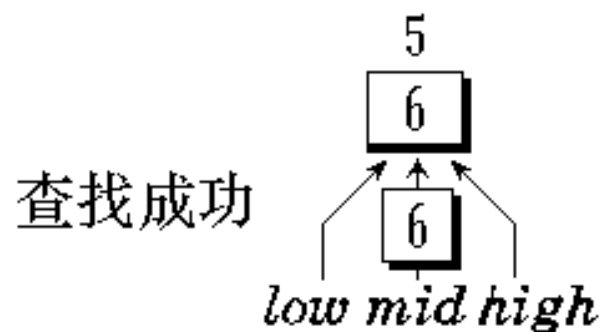
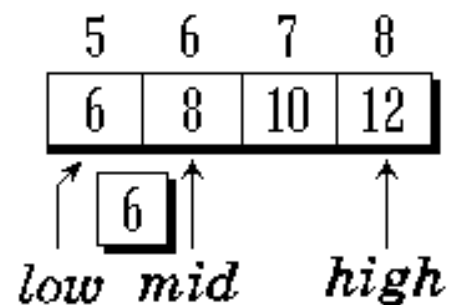
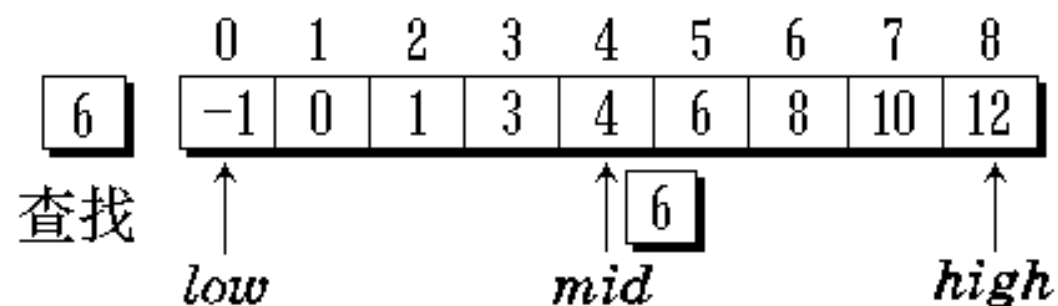
折半查找法基本过程是：

- 将表中间位置记录的关键字与查找关键字比较，如果两者相等，则查找成功；
- 否则利用中间位置记录将表分成前、后两个子表，如果中间位置记录的关键字大于查找关键字，则进一步查找前一子表，否则进一步查找后一子表。
- 重复以上过程，直到找到满足条件的记录，使查找成功，或直到子表不存在为止，此时查找不成功。

8.2.2 有序表的查找

- 设 n 个数据元素（或记录）存放在一个有序顺序表中，并按从小到大排序
- 折半查找时，先求位于查找区间正中的数据元素的下标 mid ，用其记录的值与给定值 key 比较
 1. $key == elem[mid]$ 查找成功
 2. $key < elem[mid]$ 把查找区间缩小到表的前半部分，继续折半查找
 3. $key > elem[mid]$ 把查找区间缩小到表的后半部分，继续折半查找
- 如果查找区间已缩小到没有元素（记录），仍未找到想要查找的元素（记录），则查找失败

8.2.2 有序表的查找



8.2.2 有序表的查找

- 折半查找过程可用一个称为判定树的二叉树描述，判定树中每一结点对应表中一个记录，但结点值不是记录的关键字，而是记录在表中的位置序号。
- 根结点对应当前区间的中间记录，左子树对应前一子表，右子树对应后一子表。

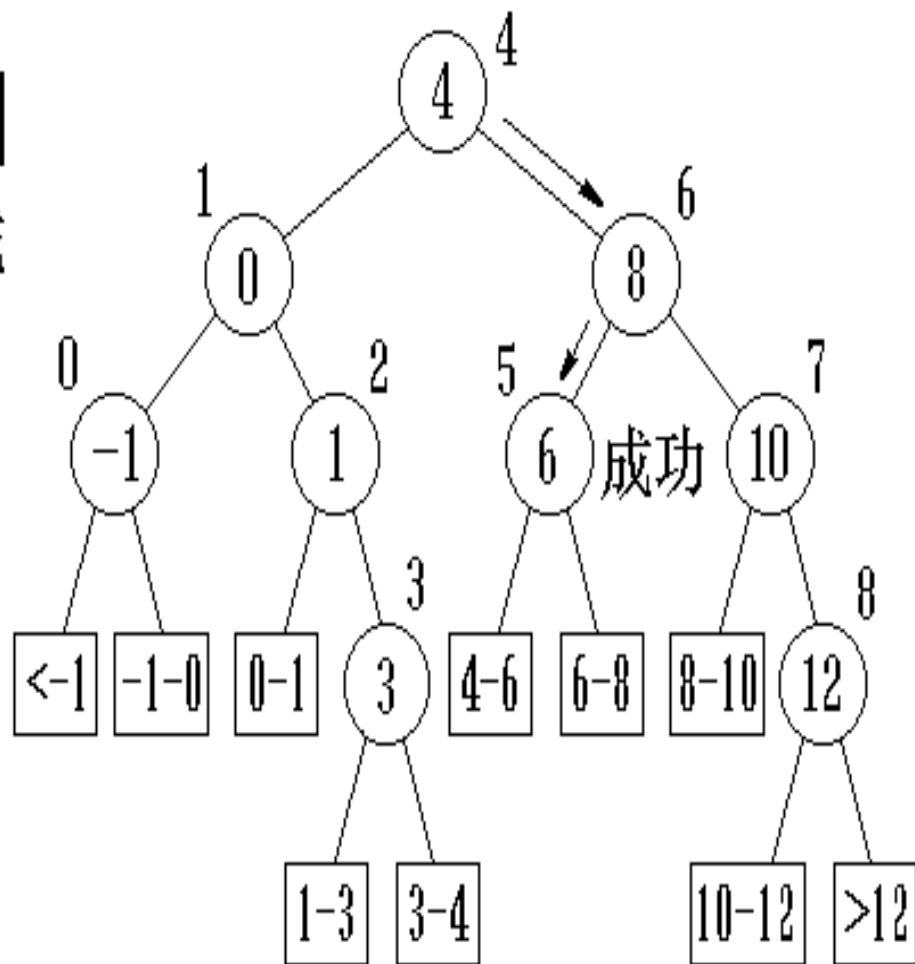
8.2.2 有序表的查找

- 这种判定树称为折半查找判定树。
- 显然，找到有序表中任一记录的过程，对应判定树中从根结点到与该记录相应的结点的路径，而所做比较的次数恰为该结点在判定树上的层次数。
- 因此，折半查找成功时，关键字比较次数最多不超过判定树的深度。
- 折半查找判定树的深度为 $\lfloor \lg n \rfloor + 1$ ，时间复杂度 $O(\lg n)$

8.2.2 有序表的查找

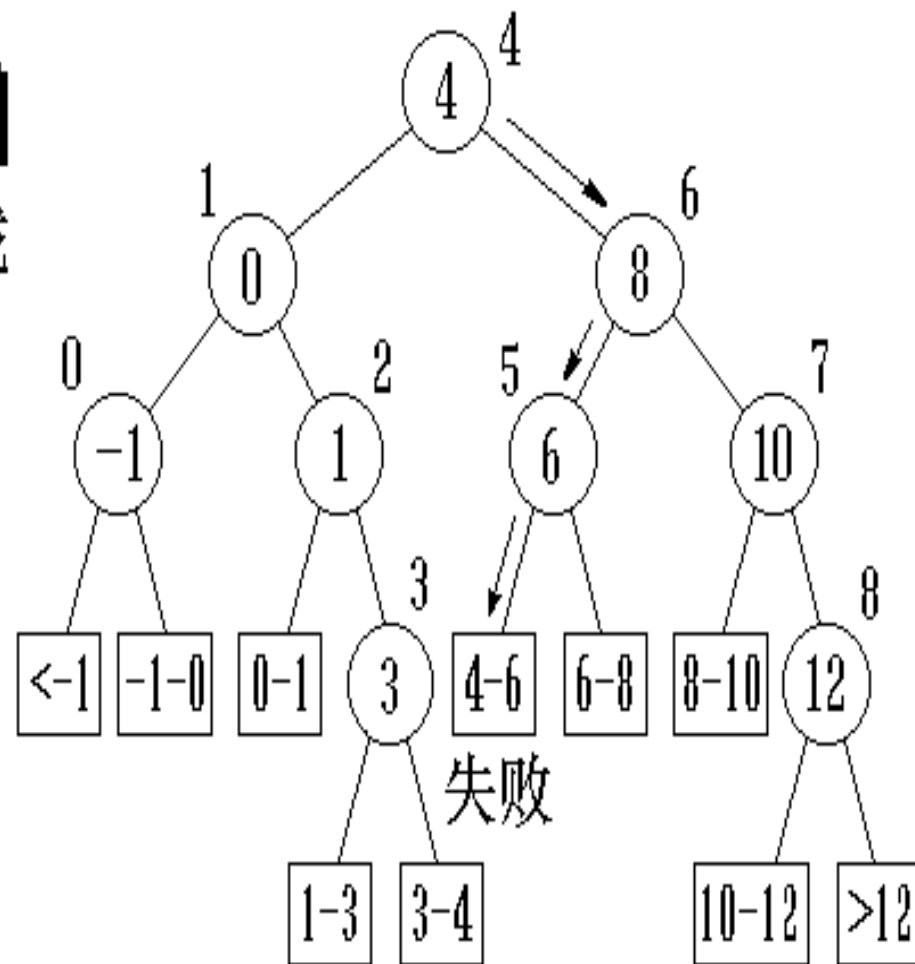
6

查找



5

查找



已知一个长度为16的顺序表L，其元素按关键字有序排列，若采用折半查找法查找一个不存在的元素，则比较次数最多是 (B)

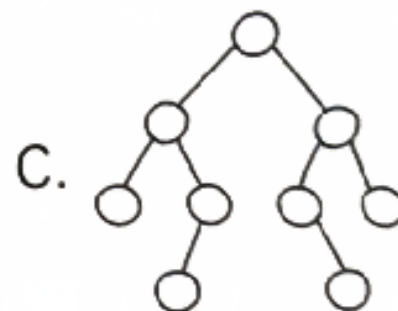
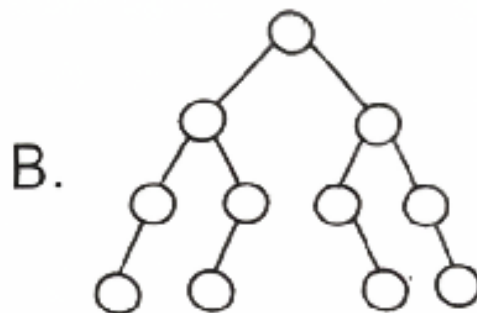
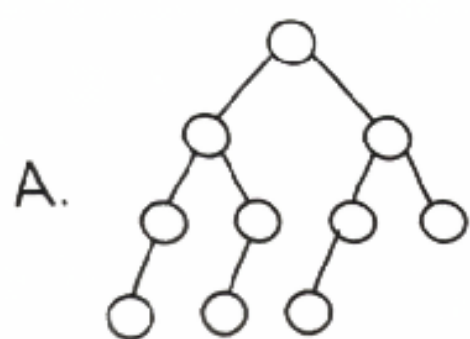
A: 4 B: 5 C: 6 D: 7

用顺序查找法在长度为n的线性表中进行查找，在等概率情况下，查找成功的平均查找长度为 (D)

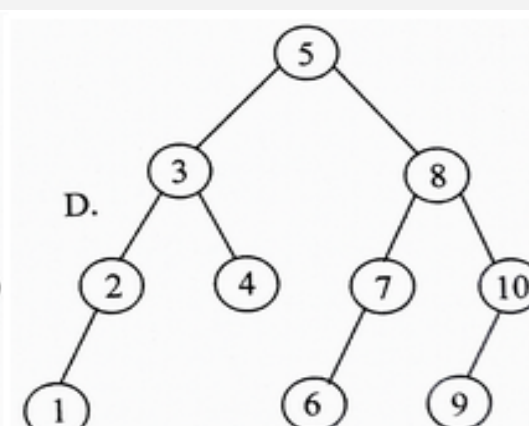
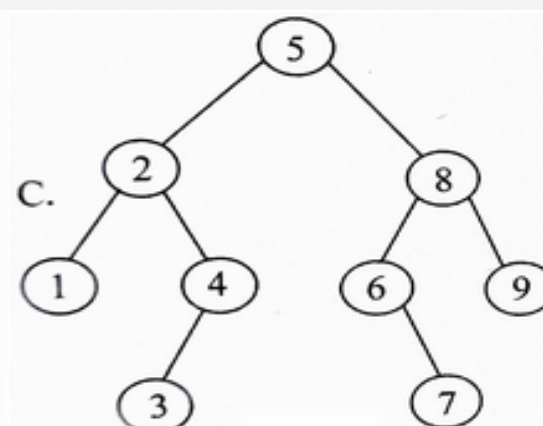
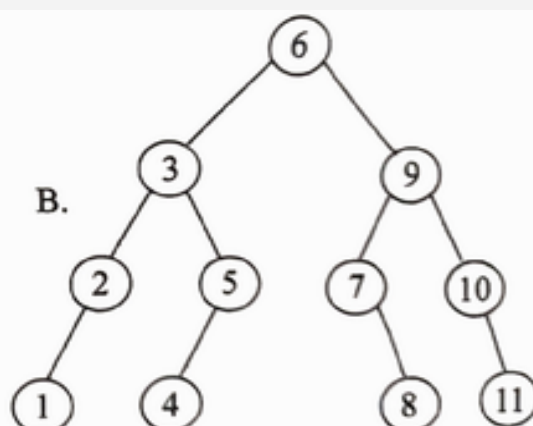
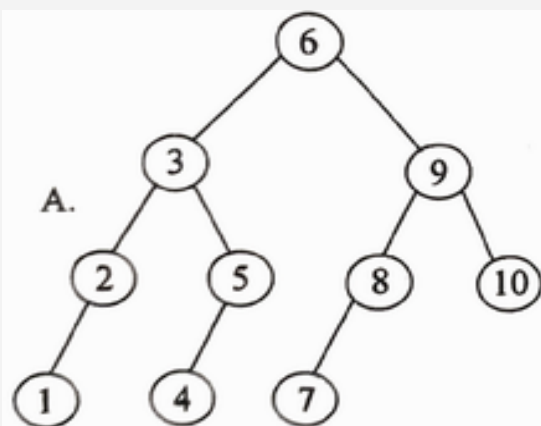
A: n B: $n/2$ C: $(n-1)/2$ D: $(n+1)/2$

解答: $(1+\cdots+n)/n$

下列二叉树中，可能成为折半查找判定树的是：



答案：A（在判定树生成中，只能遵循一个取整原则）



8.3 动态查找表

- 动态查找表的特点：表结构本身是在查找过程中动态生成的。即对给定值key，如果表中存在其关键字等于key的记录，则查找成功返回，否则插入关键字等于key的记录。
- 动态查找表也可以有不同的表示方法，我们主要讨论以树结构表示的实现方法。主要包括二叉排序树、二叉平衡树和B树等。

8.3.1 二叉排序树

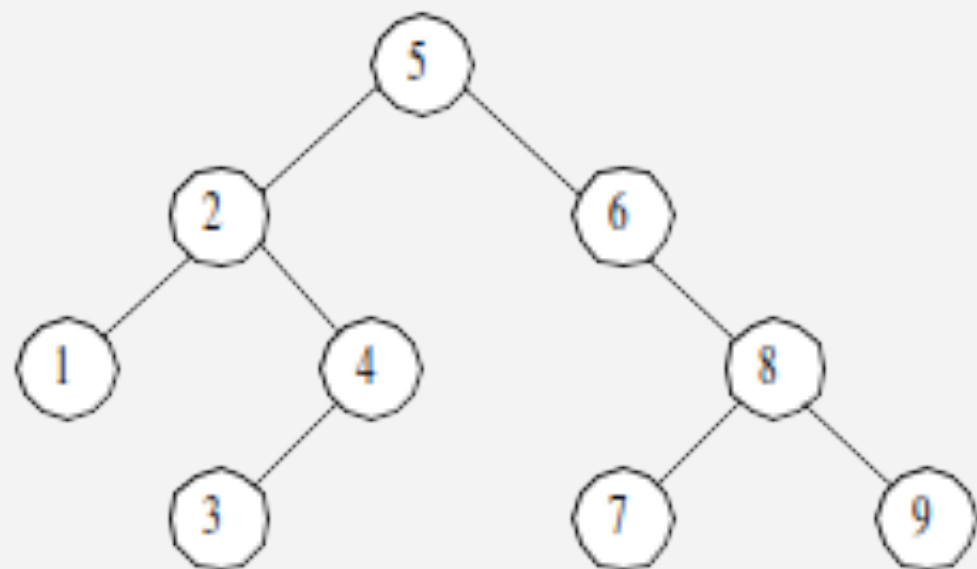
二叉排序树也称为二叉查找树，二叉排序树或者是一棵空树；或者是具有如下特性的二叉树

- 若它的左子树非空，则左子树上所有结点的值均小于根结点的值；
- 若它的右子树非空，则右子树上所有结点的值均大于根结点的值；
- 它的左右子树也分别为二叉排序树。

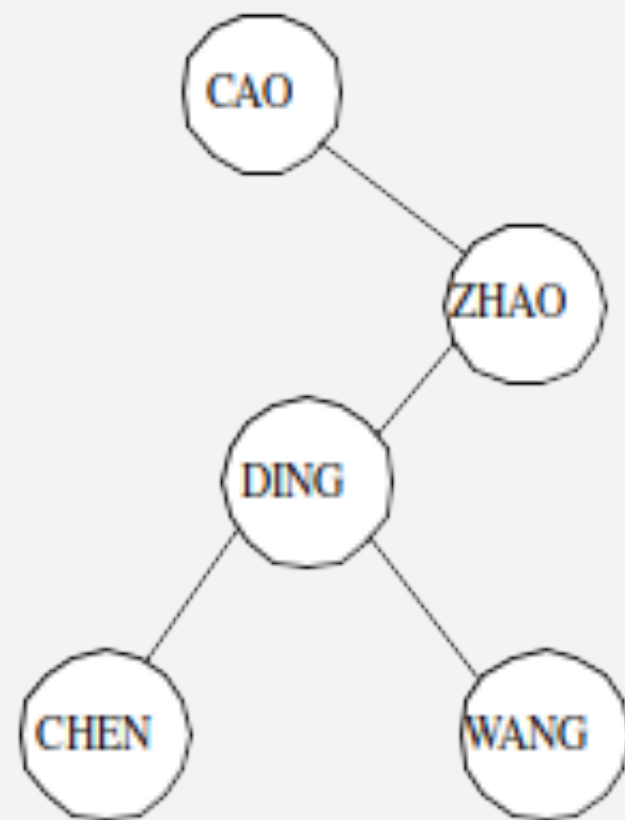
8.3.1 二叉排序树

- 如果对一棵二叉排序树进行中序遍历，可以按从小到大的顺序，将各结点关键字排列起来。
- 由此结构可知，二叉排序树的查找比较简单。

8.3.1 二叉排序树

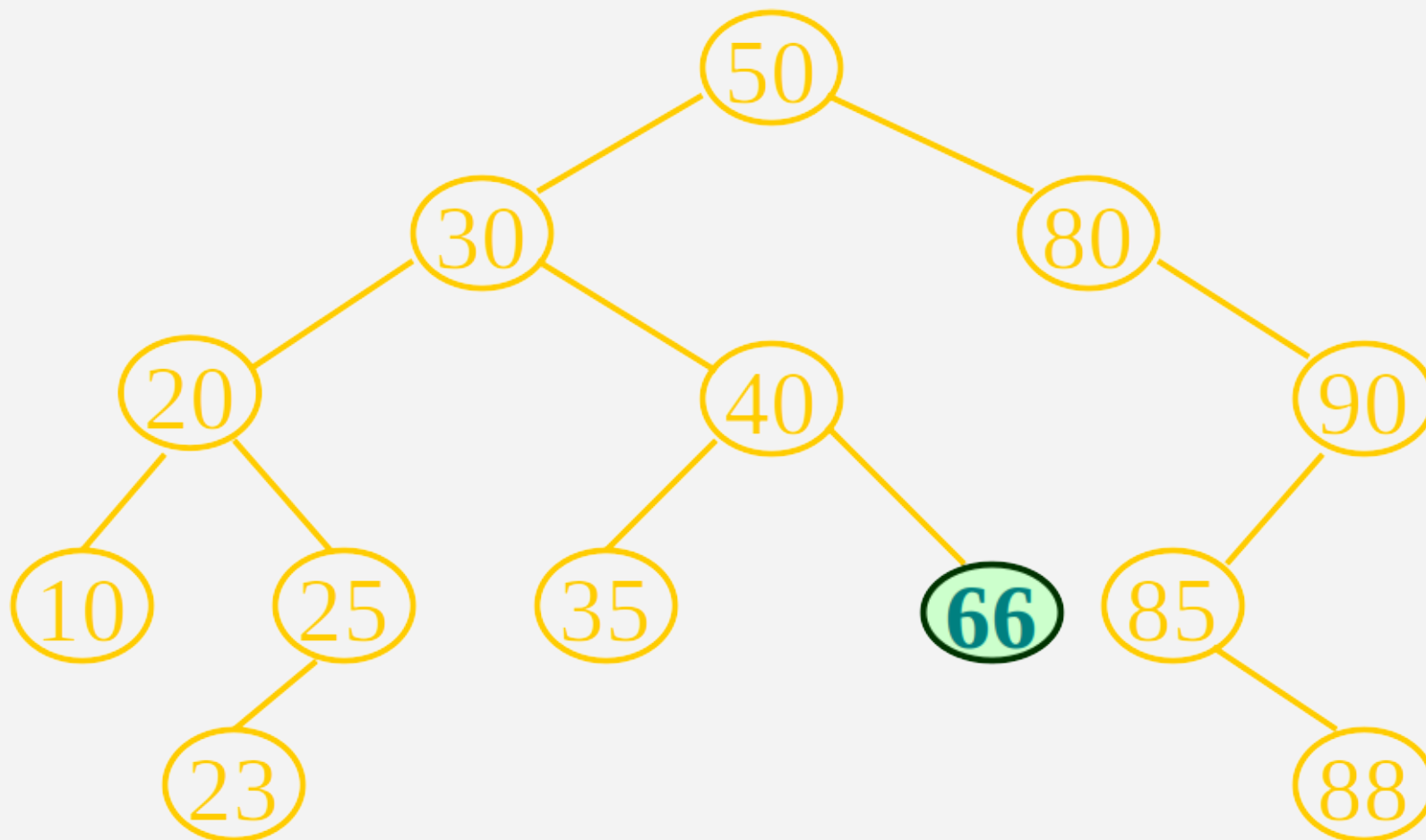


(a) 二叉排序树示例1



(b) 二叉排序树示例2(根据字符ASCII码的大小)

8.3.1 二叉排序树



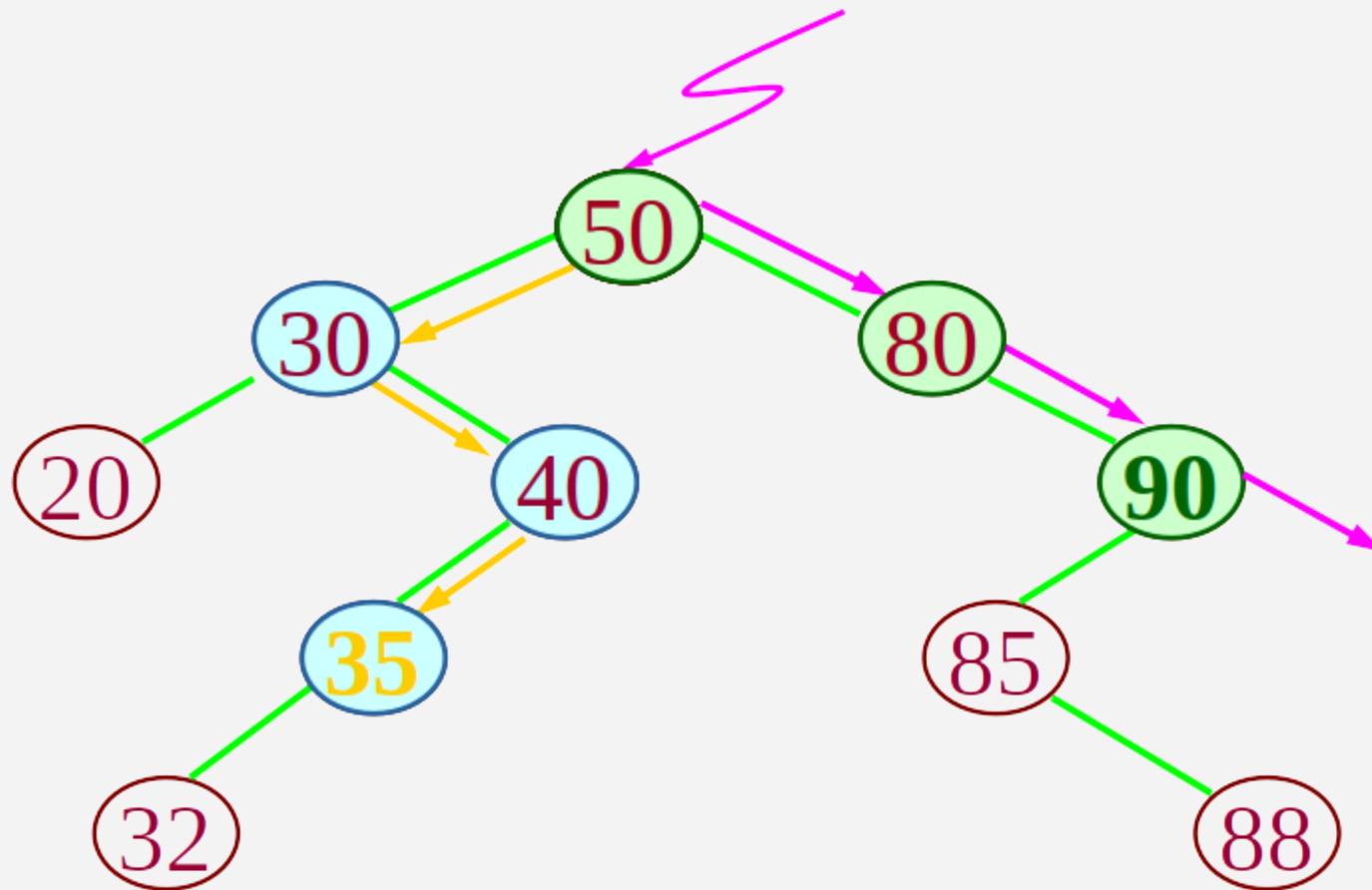
不是二叉排序树

8.3.1 二叉排序树

二叉排序树的查找：若二叉排序树为空，则查找不成功；否则

- 若给定值等于根结点的值，则查找成功
- 若给定值小于根结点的值，则继续在左子树上进行查找
- 若给定值大于根结点的值，则继续在右子树上进行查找

8.3.1 二叉排序树



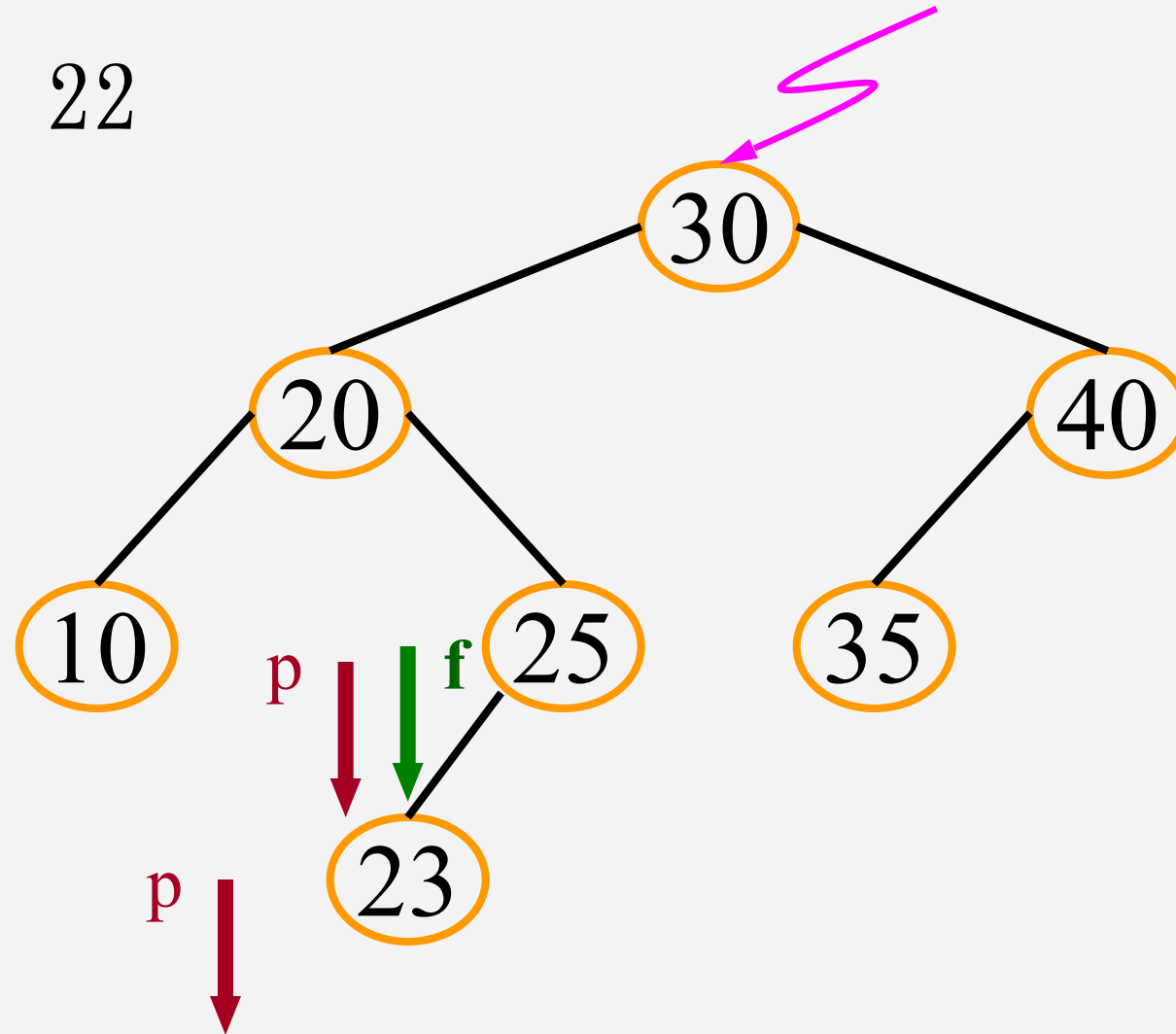
查找关键字 == 50, 35, 90, 95

8.3.1 二叉排序树

- 在查找过程中，生成了一条查找路径
- 查找成功：从根结点出发，沿着左分支或右分支逐层向下直至关键字等于给定值的结点
- 查找不成功：从根结点出发，沿着左分支或右分支逐层向下直至指针指向空树为止

8.3.1 二叉排序树

key = 22



8.3.1 二叉排序树

- 查找性能的分析：对于每一棵特定的二叉排序树，均可按照平均查找长度的定义来求它的 ASL 值。
- 显然，由值相同的 n 个关键字，构造所得的不同形态的各棵二叉排序树的平均查找长度的值不同，甚至可能差别很大

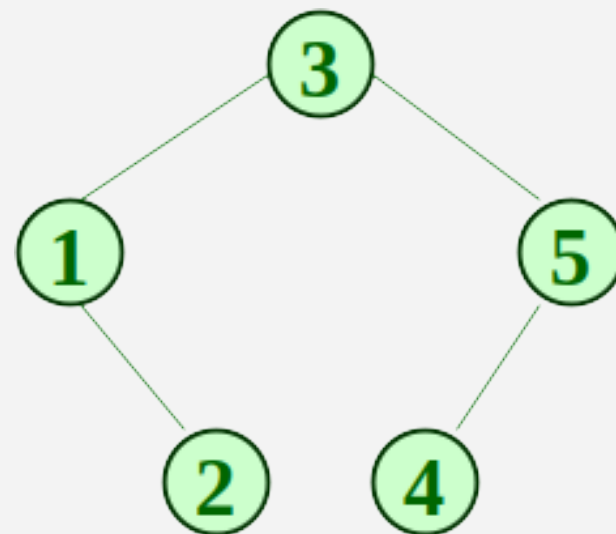
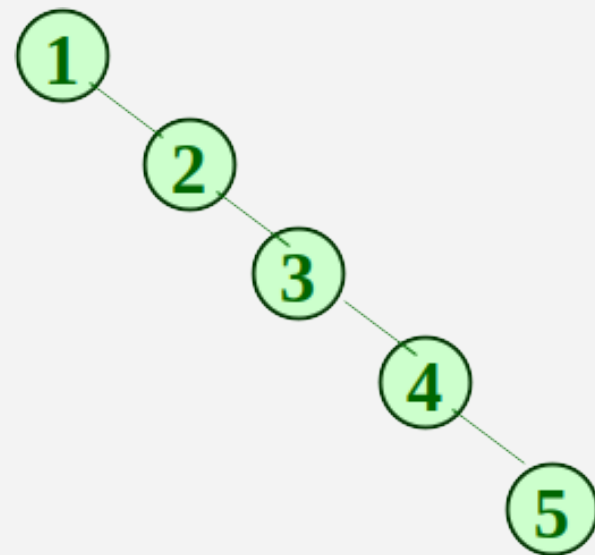
8.3.1 二叉排序树

由关键字序列 1, 2, 3, 4, 5构造而得的二叉排序树

$$ASL = (1+2+3+4+5) / 5 = 3$$

由关键字序列 3, 1, 2, 5, 4构造而得的二叉排序树

$$ASL = (1+2+3+2+3) / 5 = 2.2$$



8.3.1 二叉排序树

- 二叉排序树的插入：已知一个关键字值为 key 的结点 s ，若将其插入到二叉排序树中，只要保证插入后仍符合二叉排序树的定义即可。
- 可以推知二叉排序树的建立过程就是一个不断插入新元素的过程。

8.3.1 二叉排序树

二叉排序树的插入可以用下面的方法进行

- 若二叉排序树是空树，则key成为二叉排序树的根
- 若二叉排序树非空，则将key与二叉排序树的根进行比较，如果key的值等于根结点的值，则停止插入；如果key的值小于根结点的值，则将key插入左子树；如果key的值大于根结点的值，则将key插入右子树。

8.3.1 二叉排序树

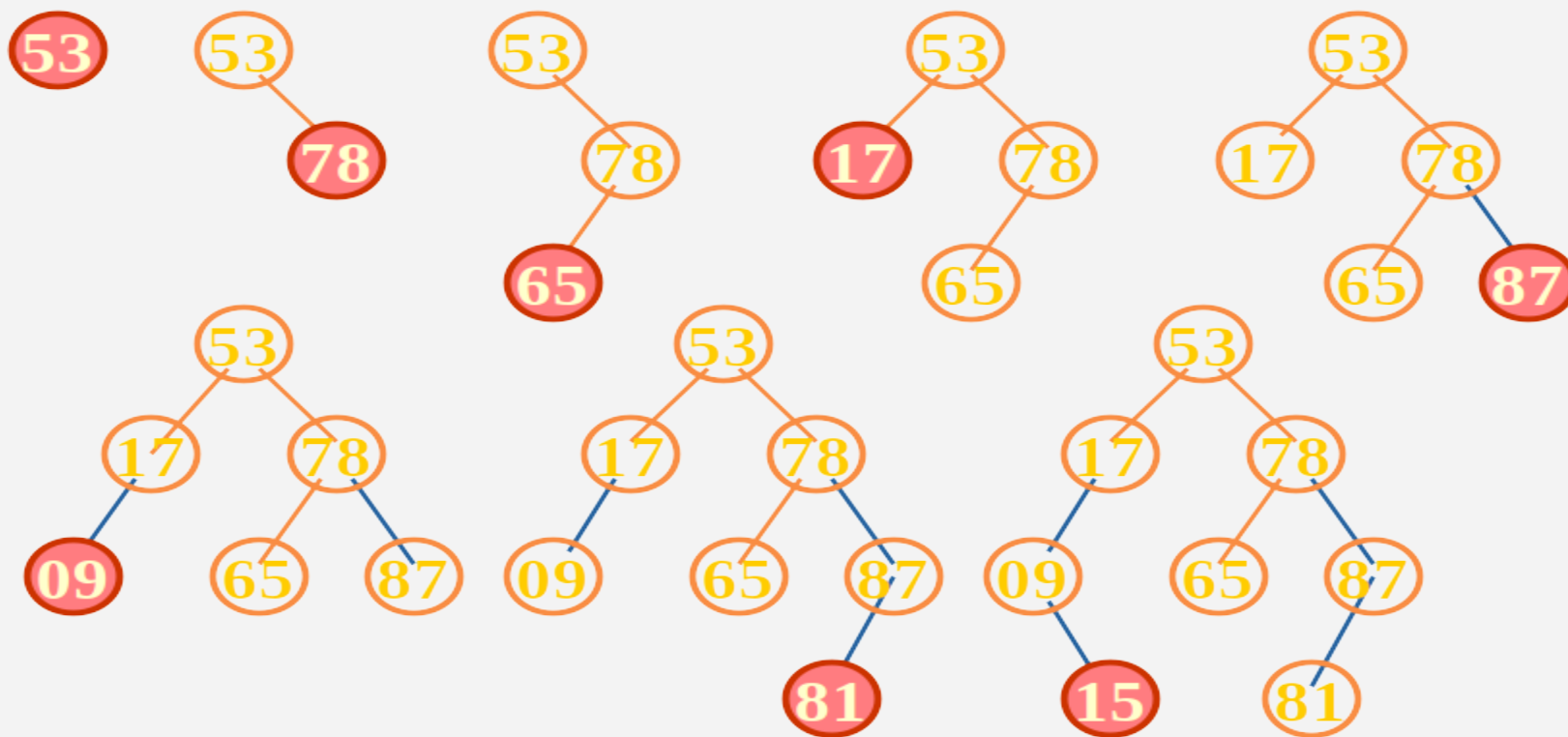
二叉排序树的建立：假若给定一个元素序列，我们可以利用上述算法创建一棵二叉排序树。

- 首先，将二叉排序树初始化为一棵空树
- 然后逐个读入元素，每读入一个元素，就建立一个新的结点并插入到当前已生成的二叉排序树中，即调用上述二叉排序树的插入算法将新结点插入。

8.3.1 二叉排序树

输入关键字，建立二叉查找树的过程

输入关键字{ 53, 78, 65, 17, 87, 09, 81, 15}



8.3.1 二叉排序树

- 同样 3 个关键字 { 1, 2, 3 }, 输入顺序不同, 建立起来的二叉查找树的形态也不同。这直接影响到二叉查找树的查找性能。
- 如果输入序列选得不好, 会建立起一棵单支树, 使得二叉查找树的高度达到最大。
- 在随机情况下, 二叉排序树的时间复杂度为 $O(\lg n)$

8.3.1 二叉排序树

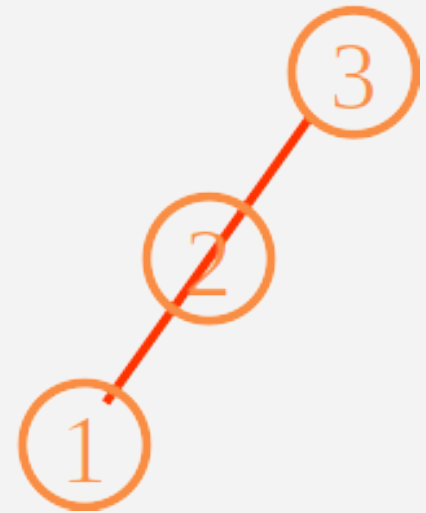
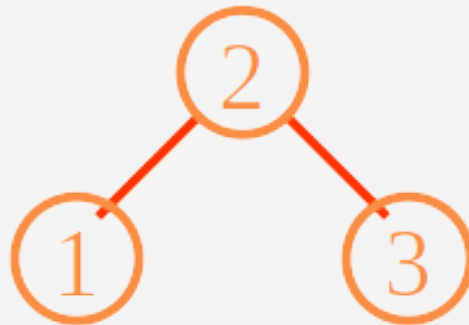
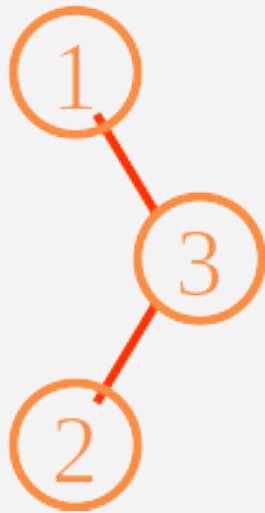
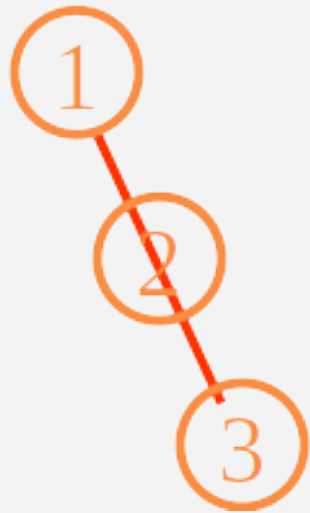
{1, 2, 3}

{1, 3, 2}

{2, 1, 3}

{3, 1, 2}

{3, 2, 1}



8.3.1 二叉排序树

二叉排序树的删除

- 从二叉排序树中删除一个结点，不能把以该结点为根的子树都删去，只能删掉该结点，并且还应保证删除后所得的二叉树仍然满足二叉排序树的性质不变。也就是说，在二叉排序树中删去一个结点相当于删去有序序列中的一个结点。

8.3.1 二叉排序树

二叉排序树的删除

- 删除操作首先要查找，已确定被删结点是否在二叉排序树中。若不在，则不做任何操作；
- 否则，假设要删除的结点为 p ，结点 p 的双亲结点为 f ，并假设结点 p 是结点 f 的左孩子（右孩子的情况类似）。分3种情况讨论

8.3.1 二叉排序树

二叉排序树的删除

- 若 p 为叶子结点，则可直接将其删除
- 若 p 结点只有左子树，或只有右子树，则可将 p 的左子树或右子树直接改为其双亲结点 f 的左子树
- 若 p 结点有非空左子树和非空右子树，则可以有两种方法处理

8.3.1 二叉排序树

二叉排序树的删除

方法1:

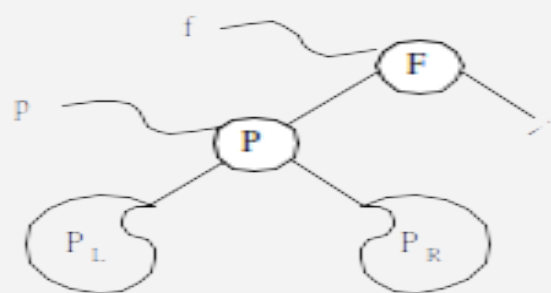
- 首先找到p结点在中序序列中的直接前驱s结点，如图 (b) 所示
- 然后将p的左子树改为f的左子树，而将p的右子树改为s的右子树：
`f->lchild=p->lchild; s->rchild= p->rchild; free(p)`
- 结果如图 (c) 所示。

8.3.1 二叉排序树

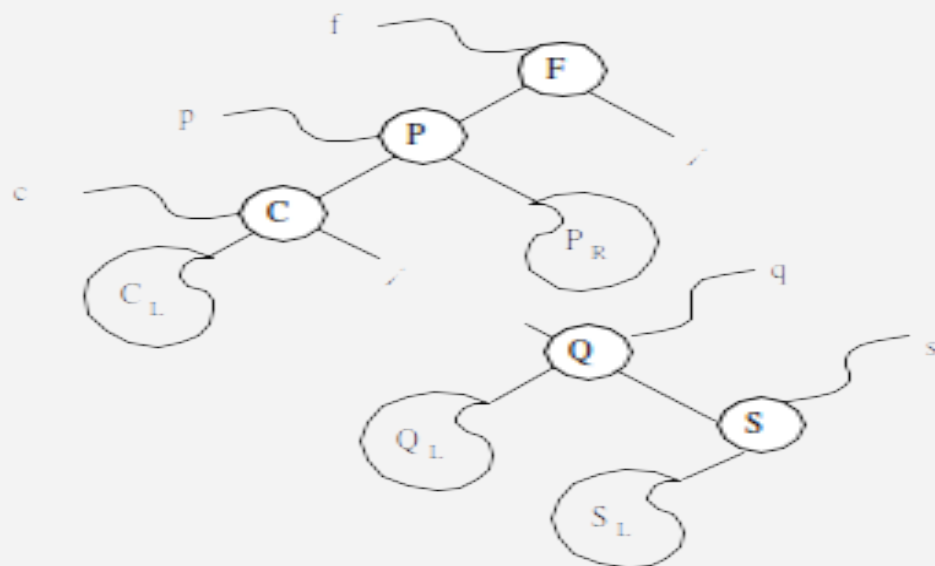
二叉排序树的删除

方法2:

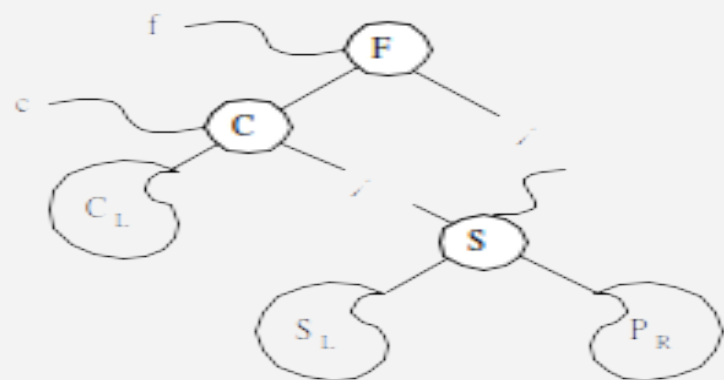
- 首先找到p结点在中序序列中的直接前驱s结点，如图（b）所示
- 然后用s结点的值替代p结点的值，再将s结点删除，原s结点的左子树改为s的双亲结点q的右子树：
`p->data=s->data; q->rchild= s->lchild; free(s)`
- 结果如图（d）所示。



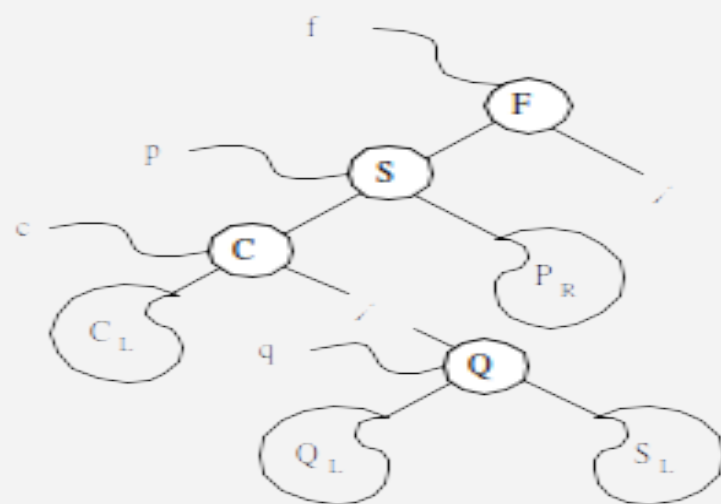
(a) P 的左右子树均不空



(b) S 为 P 的直接前驱



(c) 将 P 的左子树改为 F 的左子树，将 P 的右子树改为 S 的右子树



(d) 将原 P 结点的值改为 S 结点的值，并将原 S 的左子树改为 Q 的右子树

在任意一棵非空二叉排序树T1中，删除某结点v之后形成二叉排序树T2，再将v插入T2形成二叉排序树T3。下列关于T1与T3的叙述中，正确的是

- I. 若v是T1的叶结点，则T1与T3不同
 - II. 若v是T1的叶结点，则T1与T3相同
 - III. 若v不是T1的叶结点，则T1与T3不同
 - IV. 若v不是T1的叶结点，则T1与T3相同
- 仅I、III B. 仅I、IV C. 仅II、III D. 仅II、IV

答案. C

解析：在一棵二叉排序树中删除一个结点后再将此结点插入到二叉排序树中，如果删除的结点是叶子结点，那么在插入结点后，后来的二叉排序树与删除结点之前相同。如果删除的结点不是叶子结点，那么再插入这个结点后，由于新插入结点都是叶子结点，后来的二叉树一定发生变化。

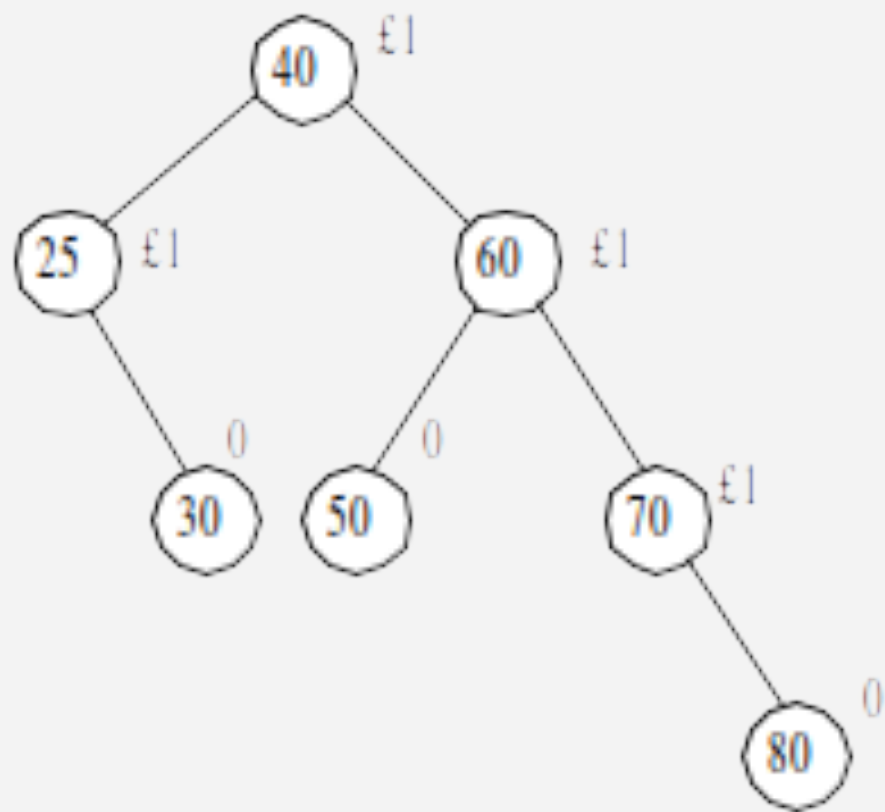
8.3.2 二叉平衡树

- A. 二叉平衡树又称为AVL树
- B. AVL树或者是空树，或者是具有下列性质的二叉排序树：
 - A. 左子树与右子树的高度之差的绝对值小于等于1
 - B. 左子树和右子树也是AVL树

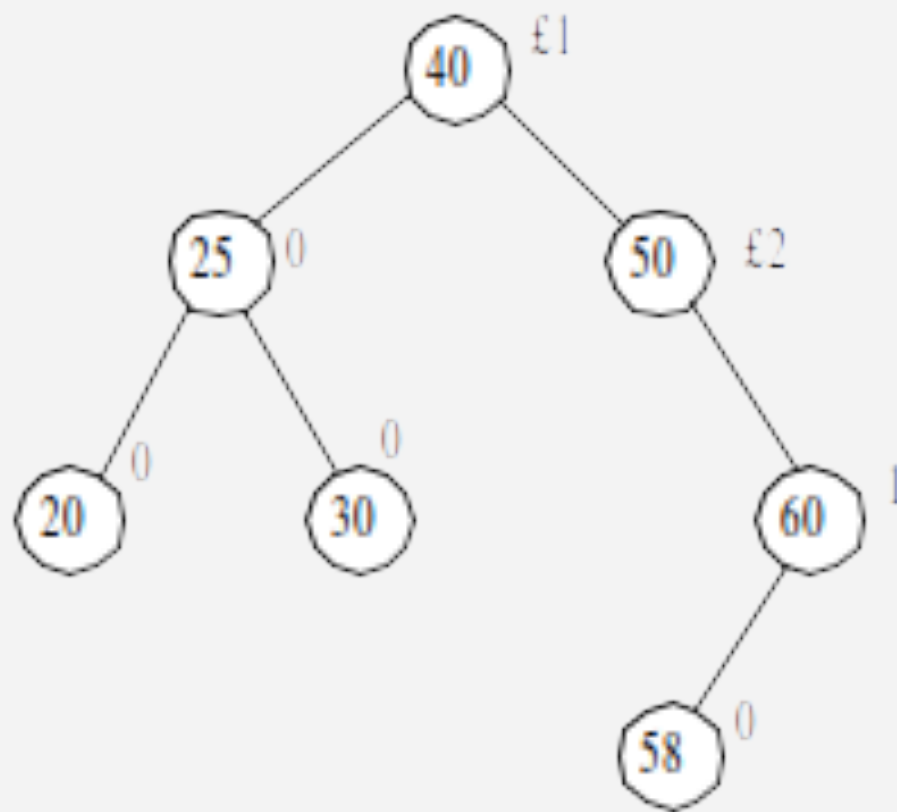
8.3.2 二叉平衡树

- A. 结点的平衡因子，其定义为：结点的左子树深度与右子树深度之差。
- B. 对一棵二叉平衡树而言，其所有结点的平衡因子只能是-1、0，或1。当我们在一个二叉平衡树上插入一个结点时，有可能导致失衡，即出现绝对值大于1的平衡因子，如2、-2。
- C. 如果一棵二叉查找树是平衡的，且有 n 个结点，平均查找长度为 $O(\log_2 n)$

8.3.2 二叉平衡树



(a) 一棵平衡二叉排序树



(b) 一棵失去平衡的二叉排序树

8.3.2 二叉平衡树

平衡化旋转

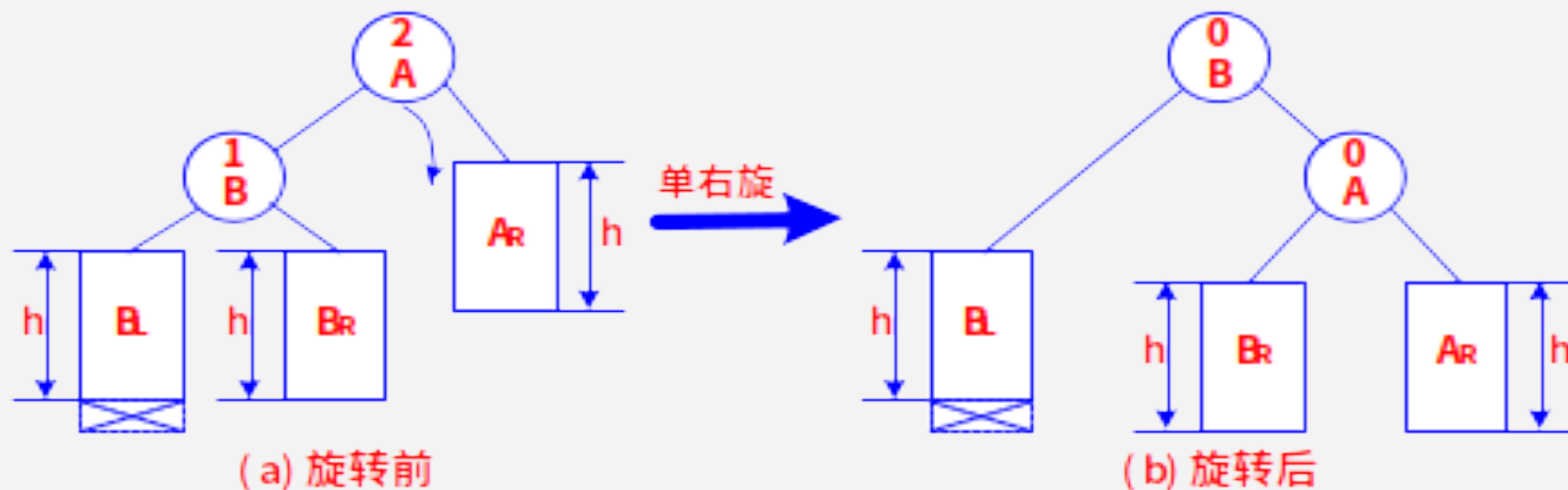
A. 如果在一棵平衡的二叉查找树中插入一个新结点，造成了不平衡。此时必须调整树的结构，使之平衡化

B. 平衡化旋转有两类

A. 单旋转（左旋和右旋）

B. 双旋转（左旋加右旋和右旋加左旋）

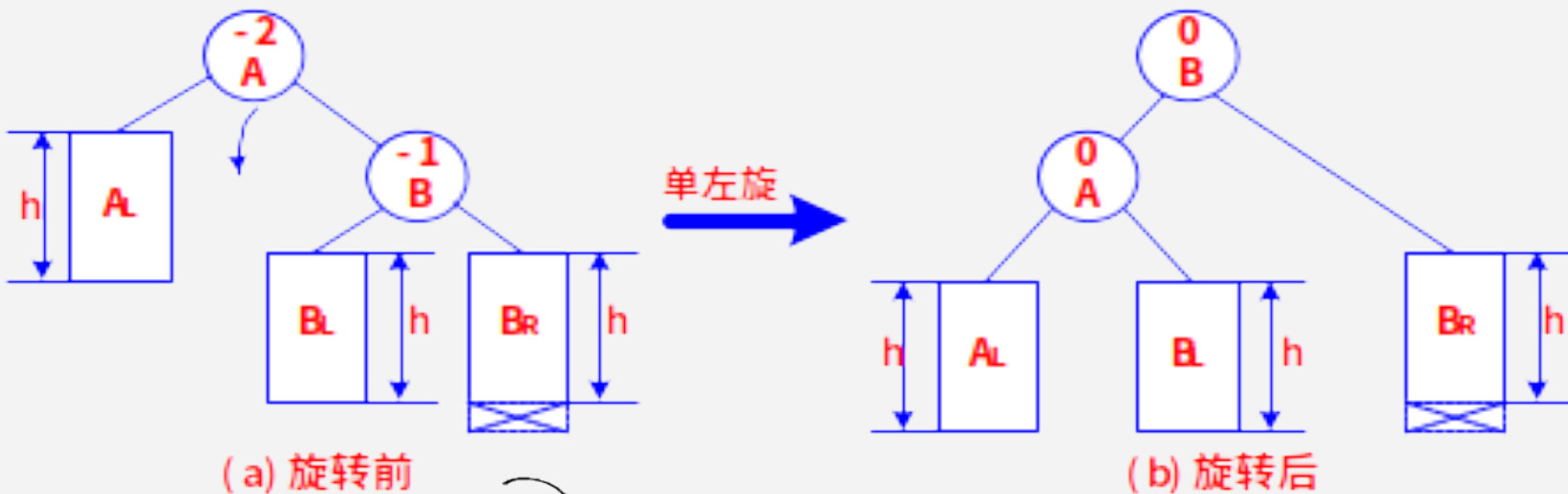
8.3.2 二叉平衡树



右单旋转LL型，插入结点在A的左孩子的左子树上
(左重 + 重子树平衡因子正 (左重孙) $\rightarrow$ LL 型)

- A. 将 B 的右子树 (B_R) “过继” 给 A 的左子树: $A.\text{left} = B_R$ (若 B_R 非空, 更新 B_R 的父节点为 A) ;
- B. 将 A “降级” 为 B 的右子树: $B.\text{right} = A$;
- C. 更新父节点指针: B 的父节点变为 A 原来的父节点, A 的父节点变为 B;
- D. 重新计算 A 和 B 的高度 (高度 = $\max(\text{左子树高度}, \text{右子树高度}) + 1$) 。

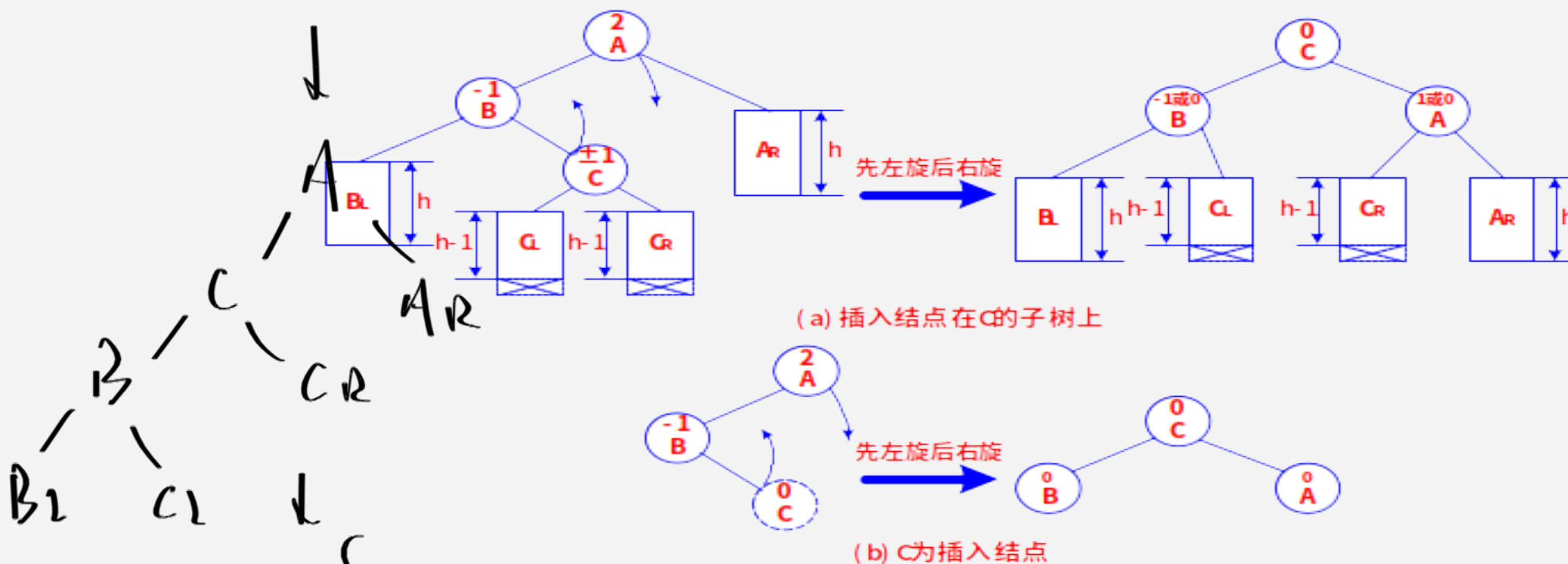
8.3.2 二叉平衡树



左单旋转RR型，插入结点在A的右孩子的右子树上
(右重 + 重子树平衡因子负 (右重孙) → RR 型)

- A. 将 B 的左子树 (B_L) “过继” 给 A 的右子树: $A.right = B_L$ (若 B_L 非空, 更新 B_L 的父节点为 A) ;
- B. 将 A “降级” 为 B 的左子树: $B.left = A$;
- C. 更新父节点指针: B 的父节点变为 A 原来的父节点, A 的父节点变为 B;
- D. 重新计算 A 和 B 的高度。

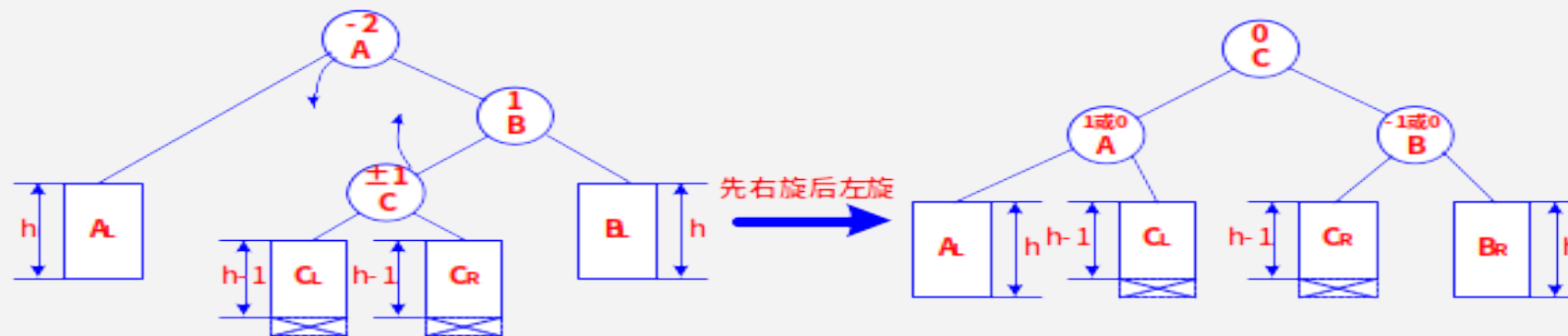
8.3.2 二叉平衡树



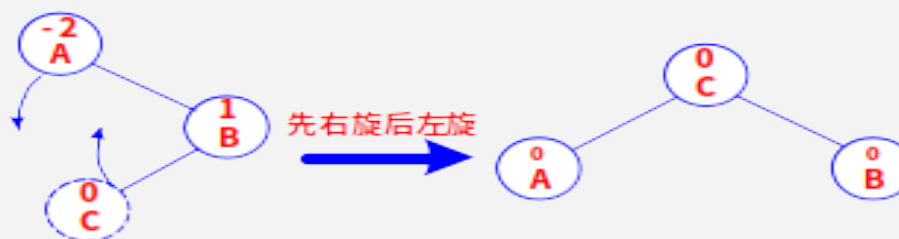
左右双旋转LR型，插入结点在A的左孩子的右子树上
(左重 + 重子树平衡因子负 (右重孙) → LR 型)

- 先对重子树根 B 做左旋 (将 C 提为 B 的父节点，转化为 LL 型)
- 再对失衡节点 A 做右旋

8.3.2 二叉平衡树



(a) 插入结点在C的子树上



(b) C为插入结点

右左双旋转RL型，插入结点在A的右孩子的左子树上
(右重 + 重子树平衡因子正 (左重孙) → RL 型)

- A. 先对重子树根 B做右旋 (将 C 提为 B 的父节点，转化为 RR 型)
- B. 再对失衡节点 A做左旋

8.3.2 二叉平衡树

综上所述， 在一个平衡二叉排序树上插入一个新结点S时，主要包括以下三步： ?

- (1) 查找应插位置，同时记录离插入位置最近的可能失衡结点A (A的平衡因子不等于0)。
- (2) 插入新结点S，并修改从A到S路径上各结点的平衡因子。
- (3) 根据A、B的平衡因子，判断是否失衡以及失衡类型， 并做相应处理。

```
// AVL树节点结构
typedef struct AVLNode {
    int val;           // 节点值
    struct AVLNode *left; // 左子树指针
    struct AVLNode *right; // 右子树指针
    int height;        // 节点高度（叶子节点高度为1）
} AVLNode;

// 辅助函数：返回两个整数的最大值
int max(int a, int b) {
    return a > b ? a : b;
}

// 辅助函数：获取节点高度（空节点高度为0）
int get_height(AVLNode *node) {
    return node == NULL ? 0 : node->height;
}

// 辅助函数：计算节点的平衡因子（左高 - 右高）
int get_balance(AVLNode *node) {
    return node == NULL ? 0 : get_height(node->left) - get_height(node->right);
}

// 辅助函数：更新节点高度（高度 = 左右子树最大高度 + 1）
void update_height(AVLNode *node) {
    if (node != NULL) {
        node->height = max(get_height(node->left), get_height(node->right)) + 1;
    }
}
```

```
// 右旋操作（修复LL型失衡，或LR型第二步）
// 参数y：失衡节点（原根），返回值：旋转后的新根
AVLNode *right_rotate(AVLNode *y) {
    AVLNode *x = y->left;    // x是y的左孩子（重子树根）
    AVLNode *T2 = x->right;   // T2是x的右子树（待过继给y）

    // 执行旋转
    x->right = y;              // y降级为x的右子树
    y->left = T2;              // T2过继给y的左子树

    // 更新y和x的高度（先更底下的y，再更x）
    update_height(y);
    update_height(x);

    return x; // x成为新根
}
```

```
// 左旋操作（修复RR型失衡，或RL型第二步）
// 参数x：失衡节点（原根），返回值：旋转后的新根
AVLNode *left_rotate(AVLNode *x) {
    AVLNode *y = x->right;    // y是x的右孩子（重子树根）
    AVLNode *T2 = y->left;    // T2是y的左子树（待过继给x）

    // 执行旋转
    y->left = x;                // x降级为y的左子树
    x->right = T2;              // T2过继给x的右子树

    // 更新x和y的高度（先更底下的x，再更y）
    update_height(x);
    update_height(y);

    return y;    // y成为新根
}
```

```
// 左旋操作（修复RR型失衡，或RL型第二步）
// 参数x：失衡节点（原根），返回值：旋转后的新根
AVLNode *left_rotate(AVLNode *x) {
    AVLNode *y = x->right;    // y是x的右孩子（重子树根）
    AVLNode *T2 = y->left;    // T2是y的左子树（待过继给x）

    // 执行旋转
    y->left = x;                // x降级为y的左子树
    x->right = T2;              // T2过继给x的右子树

    // 更新x和y的高度（先更底下的x，再更y）
    update_height(x);
    update_height(y);

    return y;    // y成为新根
}
```

```
// AVL树插入 (含平衡修复)
// 参数node: 当前子树根节点, val: 待插入值, 返回值: 插入+修复后的新根
AVLNode *avl_insert(AVLNode *node, int val) {
    // 第一步: 执行BST常规插入 (递归找到插入位置)
    if (node == NULL) { // 空节点, 创建新节点
        AVLNode *new_node = (AVLNode *)malloc(sizeof(AVLNode));
        new_node->val = val;
        new_node->left = new_node->right = NULL;
        new_node->height = 1; // 叶子节点高度为1
        return new_node;
    }

    if (val < node->val) {
        node->left = avl_insert(node->left, val); // 插入左子树
    } else if (val > node->val) {
        node->right = avl_insert(node->right, val); // 插入右子树
    } else {
        return node; // 不允许重复值, 直接返回
    }

    // 第二步: 更新当前节点高度 (插入后子树高度可能变化)
    update_height(node);
}
```


// 第三步：计算平衡因子，判断是否失衡

```
int balance = get_balance(node);
```

// 第四步：根据失衡类型，执行对应的旋转修复

// 1. LL型失衡（左重 + 左重孙）

```
if (balance > 1 && val < node->left->val) {  
    printf("触发LL型失衡，对节点%d执行右旋\n", node->val);  
    return right_rotate(node);  
}
```

// 2. RR型失衡（右重 + 右重孙）

```
if (balance < -1 && val > node->right->val) {  
    printf("触发RR型失衡，对节点%d执行左旋\n", node->val);  
    return left_rotate(node);  
}
```

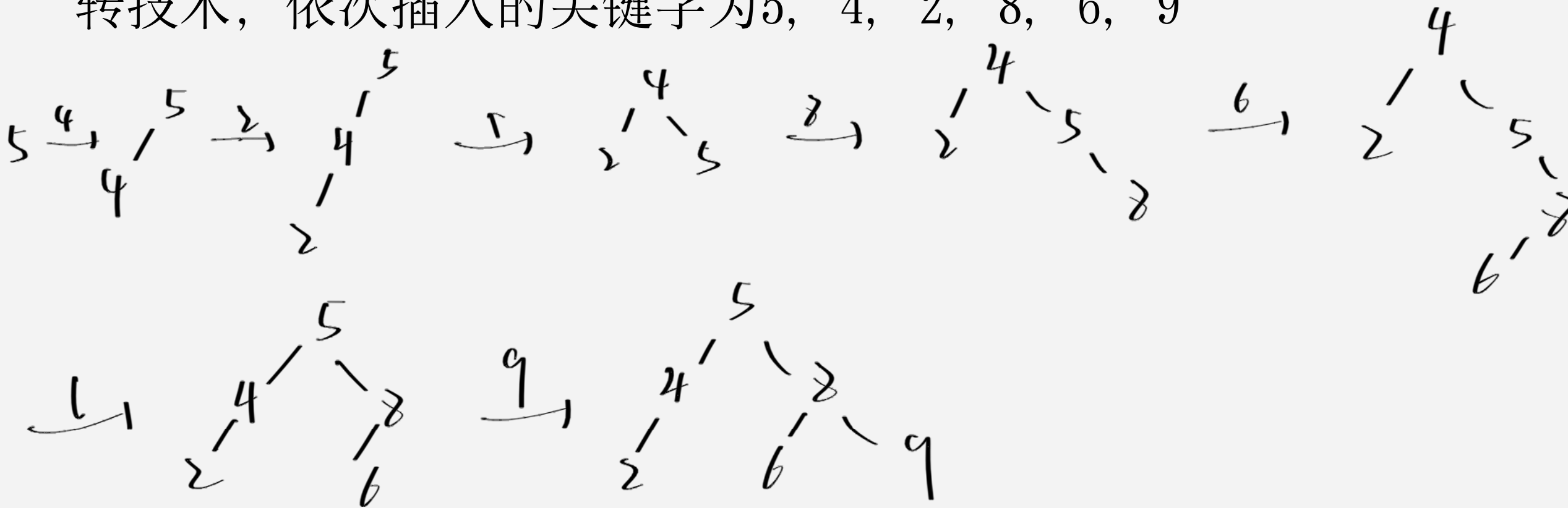
```
// 3. LR型失衡（左重 + 右重孙）：先左旋左子树，再右旋当前节点
if (balance > 1 && val > node->left->val) {
    printf("触发LR型失衡，先对节点%d执行左旋，再对节点%d执行右旋\n", node->left->val, node->val);
    node->left = left_rotate(node->left); // 左旋左子树，转化为LL型
    return right_rotate(node);          // 右旋当前节点
}

// 4. RL型失衡（右重 + 左重孙）：先右旋右子树，再左旋当前节点
if (balance < -1 && val < node->right->val) {
    printf("触发RL型失衡，先对节点%d执行右旋，再对节点%d执行左旋\n", node->right->val, node->val);
    node->right = right_rotate(node->right); // 右旋右子树，转化为RR型
    return left_rotate(node);              // 左旋当前节点
}

// 未失衡，直接返回当前节点
return node;
}
```

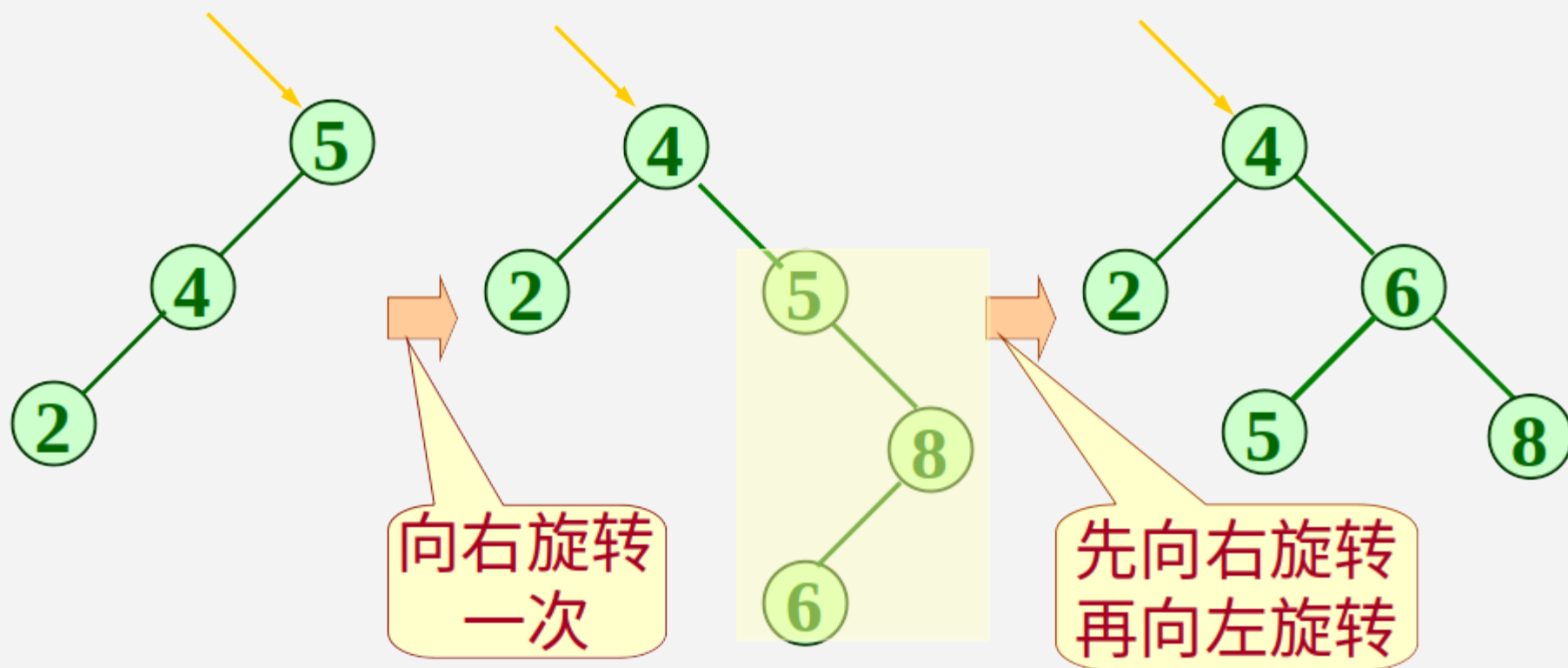
8.3.2 二叉平衡树

构造二叉平衡（查找）树的方法在插入过程中，采用平衡旋转技术，依次插入的关键字为5，4，2，8，6，9



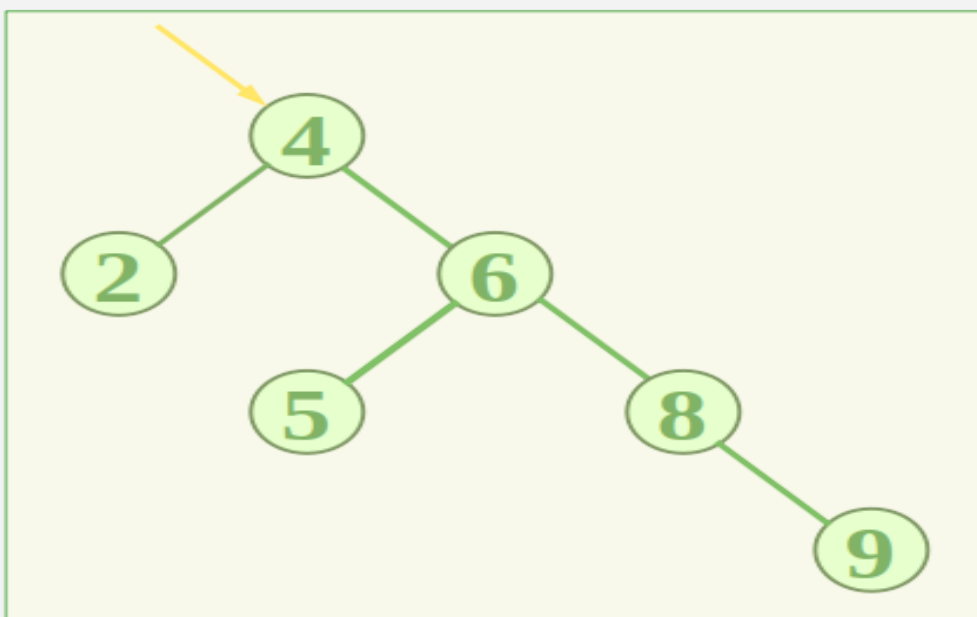
8.3.2 二叉平衡树

构造二叉平衡（查找）树的方法在插入过程中，采用平衡旋转技术，依次插入的关键字为5，4，2，8，6，9



8.3.2 二叉平衡树

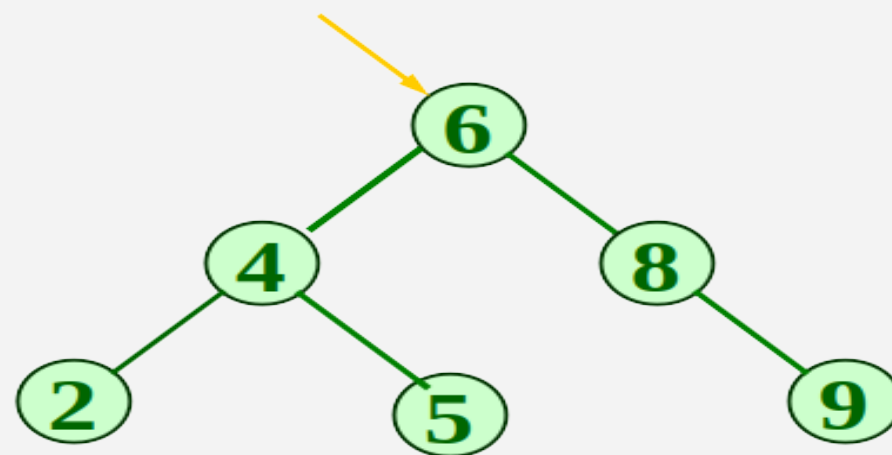
构造二叉平衡（查找）树的方法在插入过程中，采用平衡旋转技术，依次插入的关键字为5，4，2，8，6，9



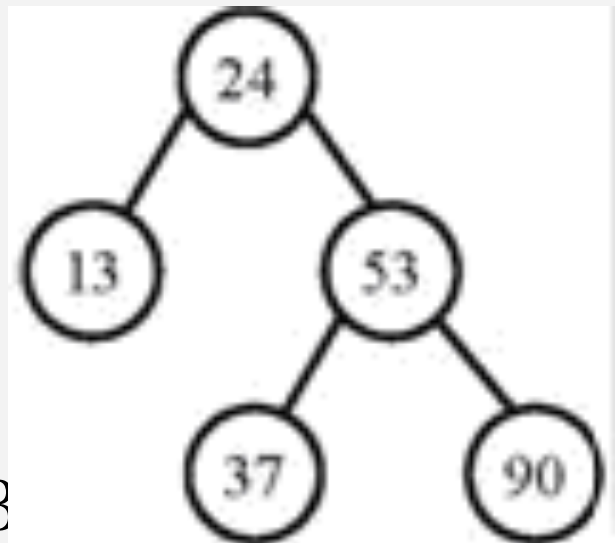
向左旋转一次



继续插入关键字 9



在下列所示的平衡二叉树中插入关键字48后得到一棵新平衡二叉树，在新平衡二叉树中，关键字37所在结点的左、右子结点中保存的关键字分别是（ C ）



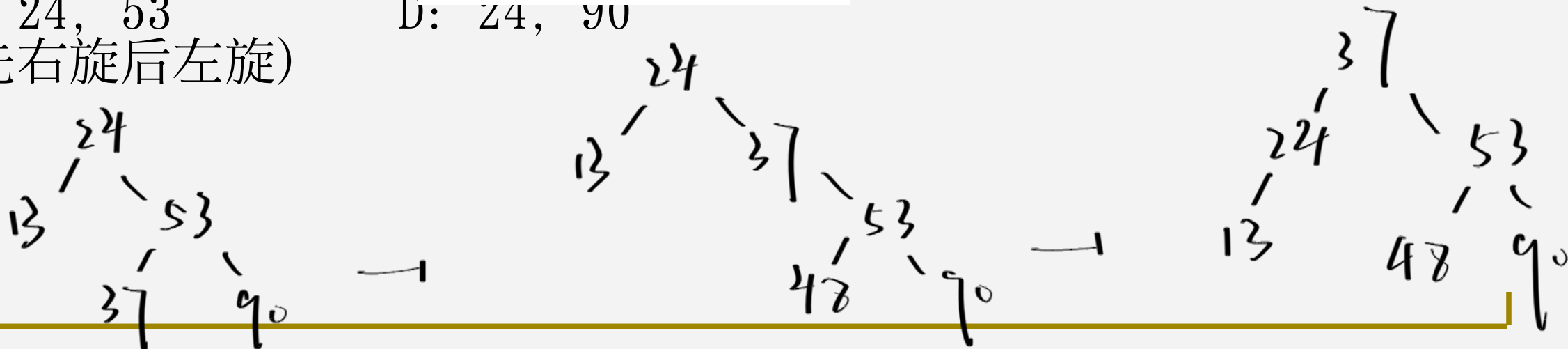
A: 13, 48

C: 24, 53

(先右旋后左旋)

B

D: 24, 90



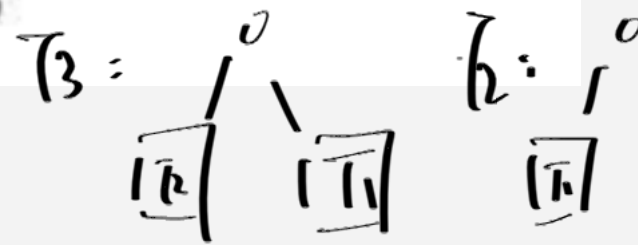
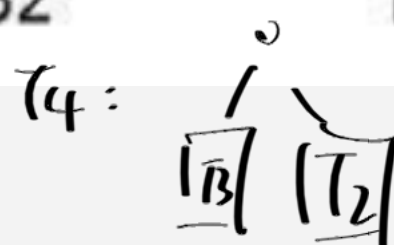
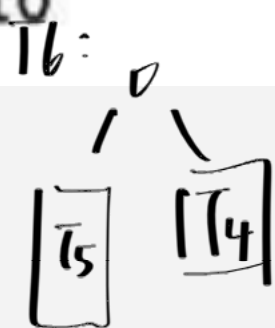
若平衡二叉树的高度为 6，且所有非叶结点的平衡因子均为 1，则该平衡二叉树的结点总数为

A. 10

B. 20

C. 32

D. 33



$$T_1 = 1$$

$$T_2 = T_1 + T_0 + 1 = 1 + 0 + 1 = 2$$

$$T_3 = T_2 + T_1 + 1 = 2 + 1 + 1 = 4$$

$$T_4 = T_3 + T_2 + 1 = 4 + 2 + 1 = 7$$

$$T_5 = T_4 + T_3 + 1 = 7 + 4 + 1 = 12$$

$$T_6 = T_5 + T_4 + 1 = 12 + 7 + 1 = 20$$

参考答案: B

分析: $f_0=0$, $f_1=1$, $f_2=f_0+f_1+1=2$,

$f_3=f_1+f_2+1=4$, $f_4=f_2+f_3+1=7$, $f_5=f_3+f_4+1=12$, $f_6=f_4+f_5+1=20$

8.3.3 B树和B+树

一棵 m 阶B树是一棵平衡的 m 路查找树，它或者是空树，或者是满足如下性质的树：

- A. 根或者是一个叶结点，或者至少有两个孩子
- B. 除根结点以外的内部结点有 $\lceil m/2 \rceil$ 到 m 个孩子
- C. 所有叶结点在树结构的同一层，并且不含信息，通常称为失败结点。因此 m 阶B树总是树高平衡的。

8.3.3 B树和B+树

阶 (m) 是 B 树的核心参数, 代表节点最多拥有的子树个数。

1、节点结构: 每个节点包含:

- A. k 个关键字 (关键字按升序排列: $key_1 < key_2 < \dots < key_k$);
- B. $k+1$ 个指向子树的指针 ($p_0, p_1, \dots, p_k$);
- C. (可选) 关键字计数 k 、叶子节点标记等辅助信息。

2、关键字个数限制:

- A. 根节点: $1 \leq k \leq m-1$ (最少 1 个关键字, 最多 $m-1$ 个关键字);
- B. 非根节点 (内部节点 / 叶子节点): $\lceil m/2 \rceil - 1 \leq k \leq m-1$ ($\lceil \rceil$ 是向上取整, 即最少 “半满” 关键字);
- C. 例: $m=3$ (3 阶 B 树), 非根节点最少 1 个关键字、最多 2 个关键字;
- D. 例: $m=4$ (4 阶 B 树), 非根节点最少 1 个关键字、最多 3 个关键字。

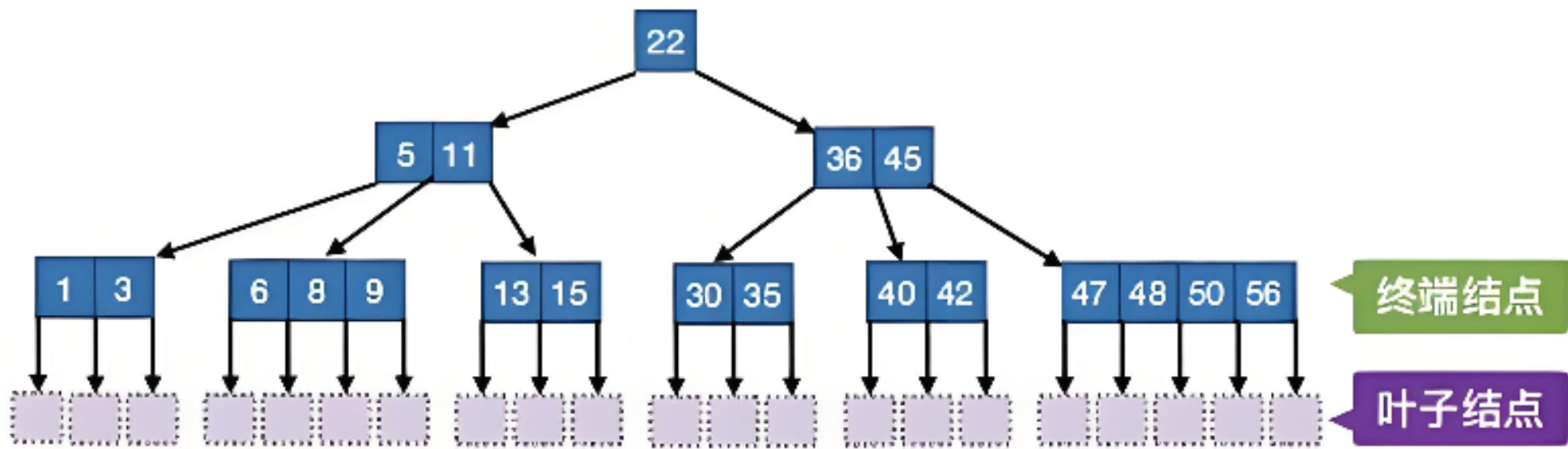
3、平衡条件: 所有叶子节点必须在同一层 (树的高度由叶子层统一决定, 无 “矮子” 叶子);

4、子树指针规则: 对于节点中第 i 个关键字 key_i :

- A. 指针 p_{i-1} 指向的子树中, 所有关键字 $< key_i$;
- B. 指针 p_i 指向的子树中, 所有关键字 $> key_i$;
- C. 叶子节点的所有子树指针均为 $null$ (或指向空)。

8.3.3 B树和B+树

B树是一种平衡的多路查找树

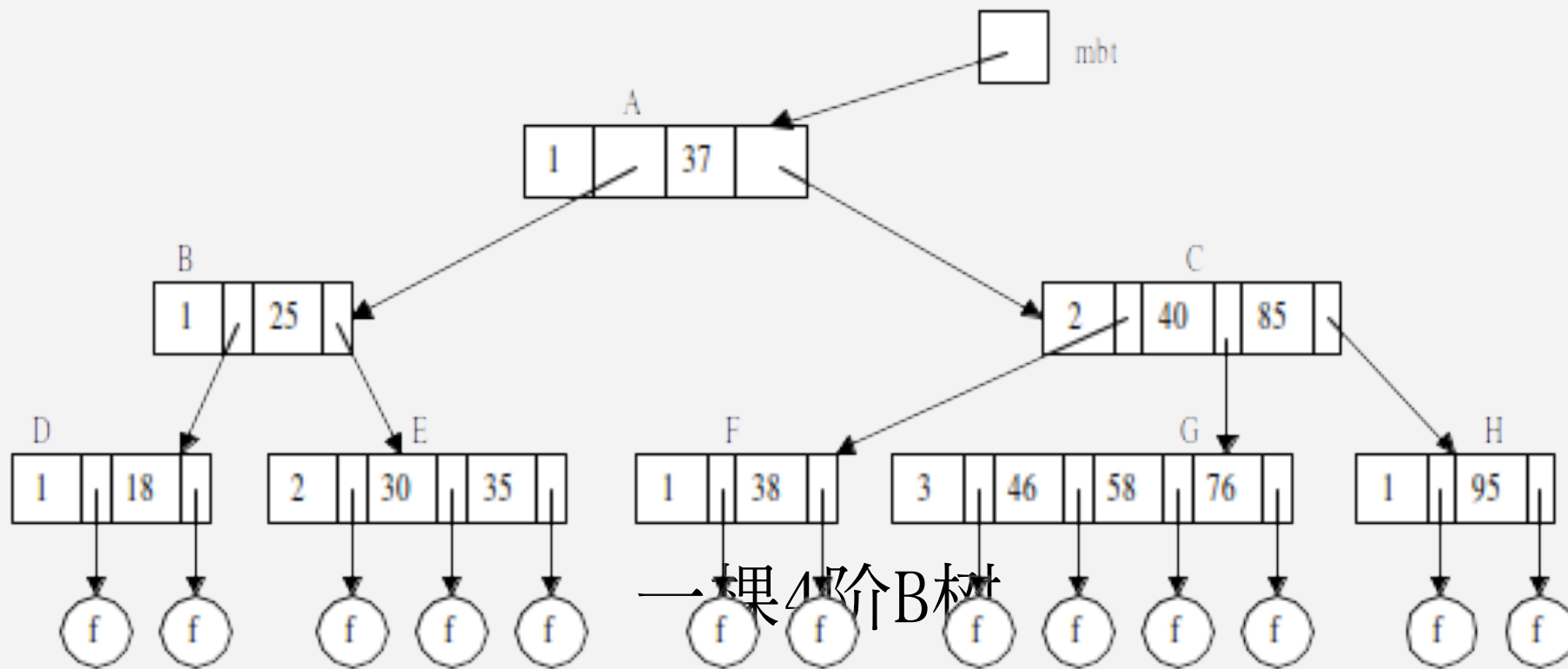


终端结点

叶子结点

8.3.3 B树和B+树

B树是一种平衡的多路查找树



一棵4阶B树

8.3.3 B树和B+树

多叉树的特点

A. 在 m 阶的B树上, 每个非终端结点可能含有: n 个关键字 K_i ($1 \leq i \leq n$) $n < m$; $n+1$ 个指向子树的指针 A_i ($0 \leq i \leq n$)

B. 如果B树的关键字个数为 n , 则叶结点个数为 $n+1$

8.3.3 B树和B+树

查找树的特点

A. 非叶结点中的多个关键字均自小至大有序排列，即：

$$K_1 < K_2 < \cdots < K_n$$

A. A_{i-1} 所指子树上所有关键字均小于 K_i

B. A_i 所指子树上所有关键字均大于 K_i

8.3.3 B树和B+树

B树的查找过程

A. 从根结点出发，沿指针查找结点和在结点内进行顺序（或折半）查找，两个过程交叉进行

- A. 若查找成功，则返回指向被查关键字所在结点的指针和关键字在结点中的位置
- B. 若查找不成功，则返回插入位置

8.3.3 B树和B+树

特性	B 树（多路平衡树）	AVL 树（二叉平衡树）
节点类型	多路节点（1 个节点存多个关键字）	二叉节点（1 个节点存 1 个关键字）
平衡条件	所有叶子在同一层	所有节点平衡因子 $\in \{-1, 0, 1\}$
树高度	低（m 越大，高度越低）	较高（二叉树，高度 $\approx \log_2 n$ ）
核心优势	减少磁盘 I/O（一次读 1 个节点 = 多个关键字）	内存中查找高效（二叉逻辑简单）
适用场景	磁盘存储（数据库、文件系统）	内存中的平衡查找场景
维护平衡的方式	节点分裂（插入）、节点合并 / 借关键字（删除）	左旋 / 右旋（插入 / 删除）

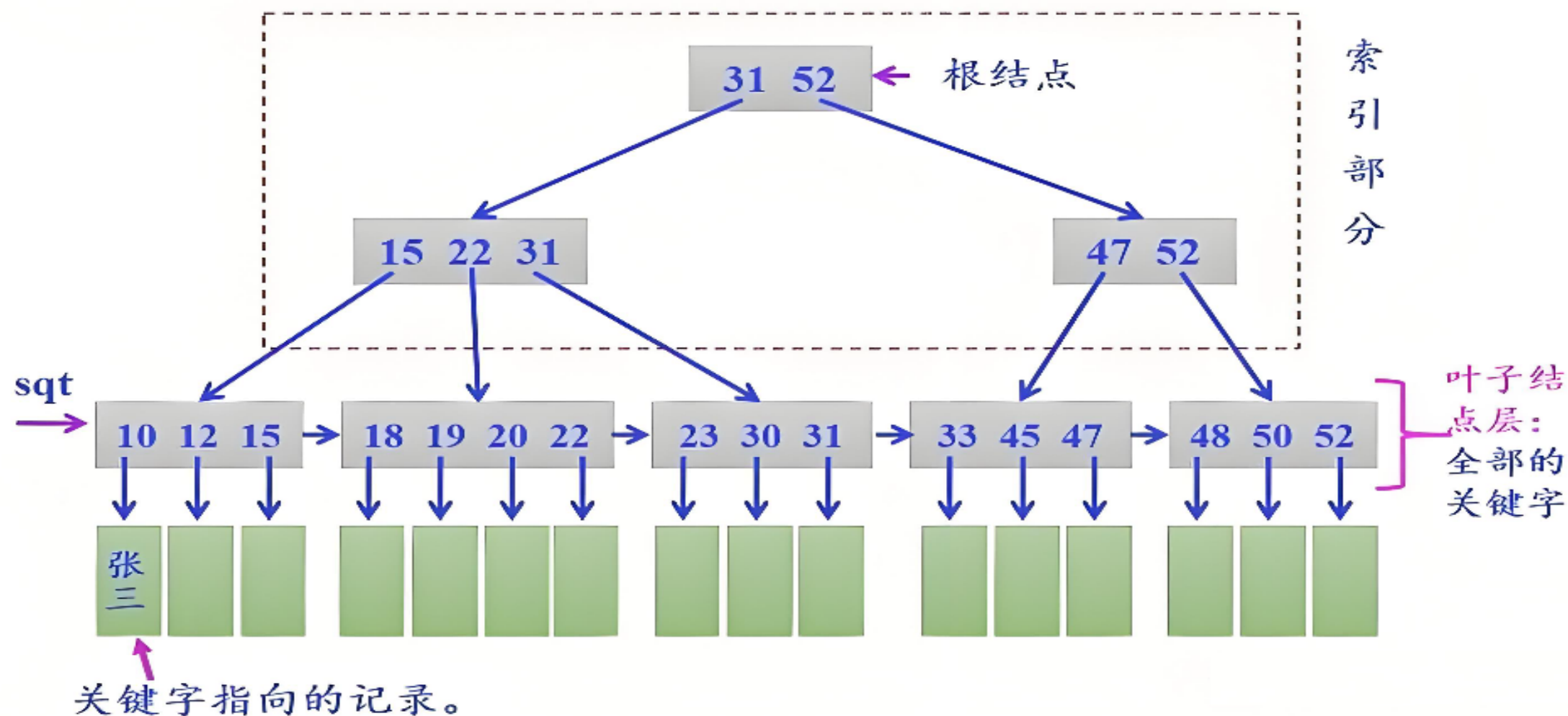
8.3.3 B树和B+树

B+树是B树的一种变形。m阶B+树有如下特征

- A. 每个结点的关键字个数与孩子个数相等，所有非最下层的内层结点的关键字是对应子树上的最大(或最小)关键字，最下层内部结点包含了全部关键字
- B. 除了根结点以外，每个内部结点有 $\lceil m/2 \rceil$ 到 m 个孩子
- C. 所有叶结点在树结构的同一层，因此树结构总是树高平衡的。

8.3.3 B树和B+树

B+树是**B**树的一些变形。一棵4阶的**B+**树示例：



8.3.3 B树和B+树

B+树的查找

- A. 在B+树上进行查找，要从根结点查找到最下层内部结点为止
- B. 也可以在最下层内部结点从左到右进行顺序查找

8.3.3 B树和B+树

特性	B 树	B + 树
关键字存储	内部节点存关键字	仅叶子节点存全部关键字，内部节点存索引关键字
叶子节点连接	无连接	叶子节点按顺序链表连接（方便范围查询）
查找路径	可能在内部节点结束	必须到叶子节点结束（路径长度一致）
范围查询	需回溯父节点	直接遍历叶子链表即可
适合场景	文件系统、部分数据库索引	关系型数据库索引（如 MySQL InnoDB）

在一株高度为3的5阶B树中，所含关键字的个数最少是
A. 5 B. 7 C. 8 D. 14

答案： A

解析： 一棵高度为2的5阶B树，根结至少 1个关键字，两个孩子至少各有2关键字。

8.4 散列表（哈希表）

A. 在线性表、树结构中查找纪录是通过与关键字的“比较”完成的。

A. 顺序查找，比较的结果为“=”或“≠”。

B. 非顺序查找，比较的结果为“<”，“=”，“>”。

B. 以上讨论的表示查找表的各种结构的共同特点：记录在表中的位置和它的关键字之间不存在一个确定的关系

8.4 散列表（哈希表）

- A. 查找的过程为给定值依次和数据元素的关键字进行比较；
查找的效率取决于和给定值进行比较的关键字个数。
- B. 用这类方法表示的查找表，其平均查找长度都不为零。

8.4 散列表（哈希表）

- A. 不同的表示方法，其差别仅在于：关键字和给定值进行比较的顺序不同。
- B. 对于频繁使用的查找表，希望 $ASL = 0$ ，只有一个办法：预先知道所查关键字在表中的位置。即，要求：记录在表中位置和其关键字之间存在一种确定的关系，这样就引出了散列表的概念。

8.4.1 散列表的概念

- A. 根据记录的关键字直接找到记录的存储位置，即为关键字和记录的存储位置建立一个函数关系，使每个关键字和结构中一个唯一的存储位置相对应。
- B. 以 $H(\text{key})$ 作为关键字为 key 的记录在表中的位置，通常称这个函数 $H(\text{key})$ 为散列函数，也称为哈希函数，按该思想建立的表为散列表。

8.4.1 散列表的概念

- A. 哈希法又称散列法、杂凑法或关键字地址计算法等，相应的表称为哈希表。
- B. 创建哈希表时，把关键字为 k 的元素直接存入地址为 $H(k)$ 的单元；以后当查找关键字为 k 的元素时，再利用哈希函数计算出该元素的存储位置 $p=H(k)$ ，从而达到按关键字直接存取元素的目的。

8.4.1 散列表的概念

- A. 哈希函数通过函数值将关键字的集合对应到某个地址集合上，它的设置很灵活，只要这个地址集合的大小不超出允许范围即可。
- B. 当关键字集合很大时，关键字值不同的元素可能会映射到哈希表的同一地址上。

8.4.1 散列表的概念

- A. 关键字值不同的元素映射到哈希表的同一地址上，即 $k_1 \neq k_2$ ，但 $H(k_1) = H(k_2)$ ，这种现象称为冲突，此时称 k_1 和 k_2 为同义词。
- B. 实际中，冲突是不可避免的，只能通过改进哈希函数的性能来减少冲突。
- C. 综上所述，哈希法主要包括以下两方面的内容：
 - A. 如何构造哈希函数
 - B. 如何处理冲突

8.4.1 散列表的概念

A. 哈希表：根据设定的哈希函数 $H(\text{key})$ 和处理冲突的方法将一组关键字映像到一个有限的连续的地址集上，并以关键字在地址中的“像”作为记录在表中的存储位置，这种表便称为哈希表，这一映像过程称为哈希造表或者散列，所得存储位置称哈希地址或散列地址。

8.4.2 构造散列函数的方法

构造散列函数时的几点要求

- A. 散列函数的定义域必须包括需要存储的全部关键码，如果散列表允许有 m 个地址时，其值域必须在0到 $m-1$ 之间。
- B. 散列函数计算出来的地址应能均匀分布在整个地址空间中：若 key 是从关键字集合中随机抽取的一个关键字，散列函数应能以同等概率取0到 $m-1$ 中的每一个值。
- C. 散列函数应是简单的，能在较短的时间内计算出结果。

8.4.2 构造散列函数的方法

对数字的关键字可有下列构造方法

- A. 平方取中法
- B. 除留余数法
- C. 随机数法
- D. 直接定址法

若是非数字关键字，则需先对其进行数字化处理

8.4.2 构造散列函数的方法

平方取中法

- A. 以关键字的平方值的中间几位作为存储地址。
- B. 平方值的中间各位又能受到整个关键字中各位的影响。

8.4.2 构造散列函数的方法

除留余数法

A. 假设哈希表长为 m , p 为小于等于 m 的数, 则哈希函数为 $H(k)=k\%p$, 其中 $\%$ 为模 p 取余运算。

B. p 最好为不大于 m 的素数 或是不含 20 以下的质因子的合数

8.4.2 构造散列函数的方法

随机数法

A. 设定哈希函数为 $H(\text{key}) = \text{Random}(\text{key})$

B. 其中, $\text{Random}()$ 为伪随机函

8.4.2 构造散列函数的方法

直接定址法

A. 此类函数直接取关键字或关键字的某个线性函数值作为散列地址：

$$\text{Hash}(\text{key}) = a * \text{key} + b \quad \{ a, b \text{ 为常数} \}$$

A. 这类散列函数是一一对一的映射，一般不会产生冲突。但是，它要求散列地址空间的大小与关键字集合的大小相同。

8.4.2 构造散列函数的方法

实际造表时，采用何种构造哈希函数的方法取决于建表的关键字集合的情况（包括关键字的范围和形态），总的原则是使产生冲突的可能性降到尽可能地小。通常应考虑以下五个因素

- A. 计算哈希函数所需的时间（简单） ☐
- B. 关键字的长度 ☐
- C. 哈希表的大小 ☐
- D. 关键字的分布情况 ☐
- E. 记录查找的频率

8.4.3 处理冲突的方法

“处理冲突”的实际含义是为产生冲突的地址寻找下一个哈希地址

A. 开放定址法

B. 链地址法

8.4.3 处理冲突的方法

开放定址法

这种方法也称再散列法，其基本思想是：当关键字key的哈希地址 $p = H(\text{key})$ 出现冲突时，以p为基础，产生另一个哈希地址 p_1 ，如果 p_1 仍然冲突，再以p为基础，产生另一个哈希地址 p_2 ，……，直到找出一个不冲突的哈希地址 p_i ，将相应元素存入其中。这种方法有一个通用的再散列函数形式， d_i 称为增量序列

$$H_i = (H(\text{key}) + d_i) \% m, \quad i=1, 2, \dots, n$$

8.4.3 处理冲突的方法

开放定址法中增量 d_i 的两种取法

A. 线性探测再散列 $d_i = i$; 这种方法的特点是: 冲突发生时, 顺序查看表中下一单元, 直到找出一个空单元或查遍全表。

B. 随机探测再散列, d_i 是一组伪随机数列。具体实现时, 应建立一个伪随机数发生器, (如 $i = (i+p) \% m$), 并给定一个随机数做起点。

{ 19, 01, 23, 14, 55, 68, 11, 82, 36 }

$$H(\text{key}) = \text{key} \% 11$$

采用线性探测再散列处理冲突

0	1	2	3	4	5	6	7	8		
55	01	23	14	68	11	82	36	19		
1	1	2	1	3	6	2	5	1		

平均查找长度:

$$(4*1+2*2+1*3+1*5+1*6)/9=2.44$$

8.4.3 处理冲突的方法

链地址法

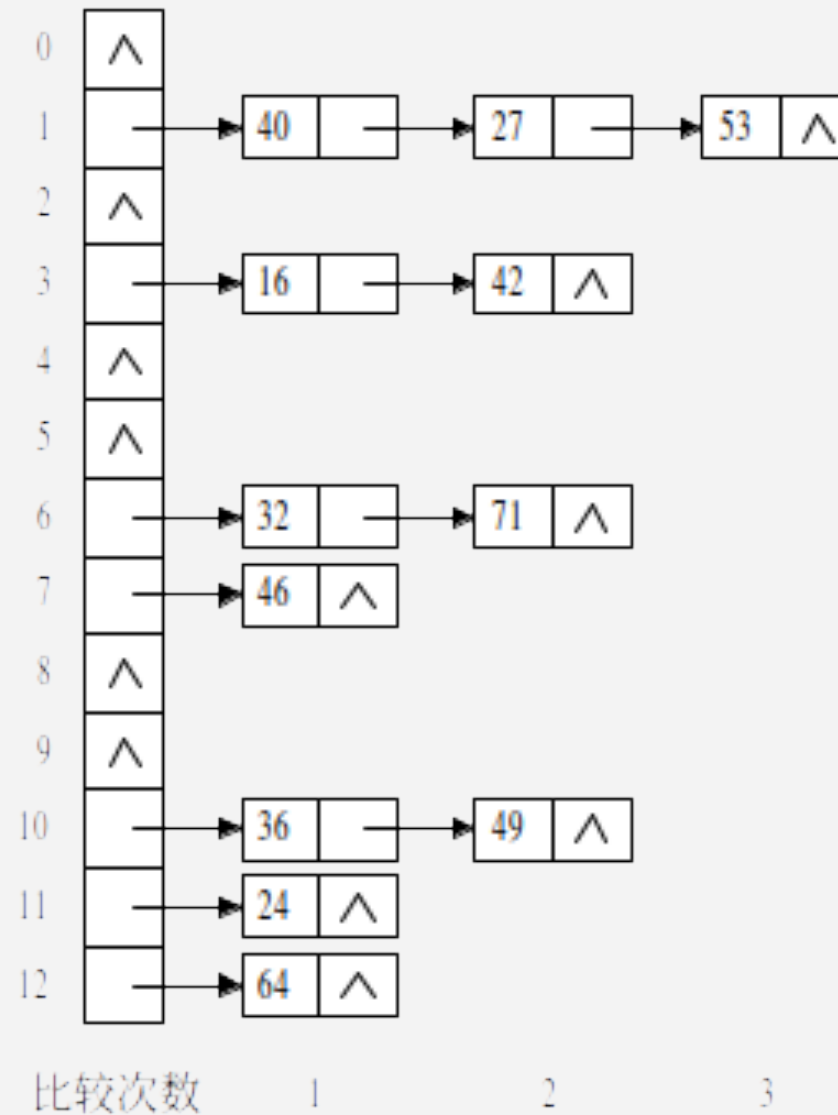
A. 这种方法的基本思想是将所有哈希地址为 i 的元素构成一个称为同义词链的单链表，并将单链表的头指针存在哈希表的第 i 个单元中，因而查找、插入和删除主要在同义词链中进行。

B. 链地址法适用于经常进行插入和删除的情况。

8.4.3 处理冲突的方法

链地址法

A. 例如，已知一组关键字 (32, 40, 36, 53, 16, 46, 71, 27, 42, 24, 49, 64)，哈希表长度为 13，哈希函数为： $H(\text{key}) = \text{key} \% 13$ ，则用链地址法处理冲突的结果如图所示。



平均查找长度: $(7*1+4*2+1*3)/12=1.5$

8.4.3 处理冲突的方法

建立公共溢出区

A. 这种方法的基本思想是将哈希表分为基本表和溢出表两部分，凡是与基本表发生冲突的元素一律填入溢出表。

8.4.3 处理冲突的方法

哈希表的查找及其分析

- A. 哈希表的查找过程与哈希表的创建过程是一致的。
- B. 当查找关键字为K的元素时，首先计算 $p_0 = \text{hash}(K)$ 。如果单元 p_0 为空，则所查元素不存在；如果单元 p_0 中元素的关键字为K，则找到所查元素；
- C. 否则重复下述解决冲突的过程：按解决冲突的方法，找出下一个哈希地址 p_i ，如果单元 p_i 为空，则所查元素不存在；如果单元 p_i 中元素的关键字为K，则找到所查元素。

8.4.3 处理冲突的方法

哈希表的查找及其分析

A. 是由于冲突的存在，哈希法仍需进行关键字比较，因此仍需用平均查找长度来评价哈希法的查找性能。哈希法中影响关键字比较次数的因素有三个：哈希函数、处理冲突的方法以及哈希表的装填因子。

B. 是哈希表的装填因子 α 的定义如下

$$\alpha = \frac{\text{哈希表中元素个数}}{\text{哈希表的长度}}$$

将关键字序列（7、8、11、18、9、14、30）散列存储到散列列表中，散列表的存储空间是一个下标从0开始的一个一维数组，散列函数为： $H(\text{key}) = (\text{key} \times 3) \text{ MOD } T$ （ T 为表长），处理冲突采用线性探测再散列法，要求装填（载）因子为0.7

问题：

- （1）请画出所构造的散列表；
- （2）分别计算等概率情况下，查找成功和查找不成功的平均查找长度。

解答：

- （1）由装载因子0.7，数据总数7个 $\rightarrow$ 存储空间长度为10 $\rightarrow T=10$

所以，构造的散列表为：

下标	0	1	2	3	4	5	6	7	8	9
关键字	30	7	14	11	8	18	.	9	.	
比较次数	1	1	1	1	1	2		1		

$$H(7) = (7 \times 3) \text{ MOD } 10 = 1$$

$$H(8) = (8 \times 3) \text{ MOD } 10 = 4$$

$$H(11) = (11 \times 3) \text{ MOD } 10 = 3$$

$$H(18) = (18 \times 3) \text{ MOD } 10 = 4$$

$$H(9) = (9 \times 3) \text{ MOD } 10 = 7$$

$$H(14) = (14 \times 3) \text{ MOD } 10 = 2$$

$$H(30) = (30 \times 3) \text{ MOD } 10 = 0$$

$$(2) \text{ 查找成功的ASL} = (1+1+1+1+1+2+1) / 7 = 8/7$$

$$\text{查找不成功的ASL} = (7+6+5+4+3+2+1+2+1+1) / 10 = 3.2$$